Blockchain-assisted Multi-party Computation for Hashi

Mysten Labs
Version 1
October 2026

Abstract

Hashi employs a threshold Schnorr signing protocol in which a committee of $n = 3f+1$ parties, tolerating up to f Byzantine faults, produces standard Schnorr signatures under a single public key. The interactive preprocessing takes place in a message-independent offline phase, in which a distributed key generation (DKG) protocol establishes the joint key, dealers distribute batches of Shamir-shared nonces with publicly verifiable random-oracle consistency checks, and certified dealings are combined through a super-invertible matrix into presigning slots. After the slot assignment and the required beacon disclosures, each signature consumes one slot in a single round of partial-signature broadcasts; offsets from a message beacon and a batch beacon mask the combined nonce. Reed–Solomon decoding allows the aggregator to reconstruct the signature when up to f partial signatures are faulty. For our beacon-based solution, we prove unforgeability in the random-oracle model by a reduction to the discrete logarithm problem using a forking argument. We also show how to parameterize reconstruction/secrecy and Byzantine-fault thresholds separately in the low-threshold regime, and how to reduce stake-weighted committees to small integer weights with bounded precision loss.

1 Introduction

Threshold signatures are widely used in digital asset systems to eliminate single points of failure in key management and transaction authorization. In a threshold signature scheme, the signing key is secret-shared among n parties so that any qualified subset can jointly produce signatures under a single public key while tolerating up to f Byzantine faults. No party below the reconstruction threshold can recover the key from its shares; unforgeability and liveness hold within the stated corruption and network assumptions. Threshold signature schemes may be classified by the interaction required during signing. In non-interactive schemes such as threshold BLS [BLS01, Bol03], each party locally computes a partial signature from its share, and the partial signatures are aggregated without further coordination; however, such schemes rely on pairing-based constructions that are incompatible with the Schnorr signatures over secp256k1 used by Bitcoin. In interactive schemes such as threshold Schnorr [Sch90, SS01, KG20], signing requires coordination: each signature consumes a fresh shared nonce whose secret component must remain hidden even though its commitment enters the Fiat–Shamir challenge. Generating such nonces without leaking information about the signing key is the source of the multi-party computation (MPC)-style interaction that makes threshold Schnorr protocols complex. This interaction is expensive in practice: naive protocols require several rounds of communication per signature, nonce generation must be protected against misuse because nonce reuse or bias reveals the signing key, and a deployable protocol must preserve safety and liveness in the presence of Byzantine parties, message reordering, and committee churn.

Hashi confines this interaction to a message-independent offline phase and reduces online signing to a single round with required beacons. In the offline phase, the committee generates shared nonces in batches and combines certified dealings through a super-invertible (SI) matrix into single-use presigning slots. Each signature then consumes one slot in a single broadcast round, and error-correcting aggregation reconstructs the signature response even when up to f partial signatures are faulty. Because the interactive cost is amortized over a batch of presignatures, Hashi achieves high throughput while retaining the robustness and single-key semantics of threshold signing.

This paper presents a modular MPC signing protocol designed for robust deployment. The construction combines a distributed key generation (DKG) with a batch nonce distribution mechanism and a linear-algebraic presigning step based on a super-invertible (SI) matrix. Online signing uses one

partial-signature round after the slot assignment and two-beacon prerequisites, and an error-correcting aggregation procedure provides robustness against Byzantine faults. The security proof proceeds in the random-oracle model through a sequence of games and a forking-style reduction to the discrete logarithm problem.

The technical core of the protocol is the offline generation of shared nonces. Each committee member acts as a dealer and secret-shares a large batch of nonces in a single dealing, posting commitments together with encrypted shares as in EthDKG [SJSW19]. A random-oracle challenge folds the batch into a single masking polynomial, so that each recipient can verify the consistency of the entire dealing with one check, a technique previously used for verifiable secret sharing in [ABCP23, SS24]. A dealing cannot serve as a signing nonce on its own, because a Byzantine dealer knows the secrets it dealt. Certified dealings from different dealers are therefore combined through a super-invertible matrix, which guarantees that the resulting presigned nonces are uniformly random in the adversary's view as long as dealers of total weight at most f are Byzantine. During online signing, each signature consumes one presigned nonce, offset by the sum of values derived from the message and batch beacons, and each party broadcasts a single Schnorr partial signature. The aggregator reconstructs the signature by interpolation, resorting to Reed–Solomon decoding when some of the partial signatures are faulty.

Paper overview. Section 2 fixes notation and recalls the building blocks. Section 3 describes the AVSS layer with the completeness guarantee, used to establish the joint Schnorr verification key and Shamir shares of the signing key over weighted virtual evaluation points. The same layer supports key resharing across committee changes while preserving the public key. Section 4 gives the offline batch nonce-generation protocol, in which each dealer distributes many Shamir-shared nonce polynomials together with public commitments and random-oracle consistency checks. Section 5 shows how certified nonce sharings are combined through a super-invertible matrix into presigning slots, and how each online signing instance consumes one fresh slot, masked by public beacons, to produce a standard Schnorr signature under the joint key. Once its slot assignment and beacons are fixed, the online phase consists of one round of Schnorr partial-signature broadcasts; the aggregator reconstructs the response by interpolation and uses Reed–Solomon decoding to tolerate faulty partials. Section 6 formalizes security of the MPC signing protocol and proves unforgeability. The proof simulates the DKG, nonce-generation, presigning, and signing transcripts, and applies a forking argument to extract a discrete logarithm from a successful forgery in the random oracle model.

Section 7 presents an asynchronous verifiable information dispersal protocol that can replace reliable broadcast in the batch AVSS. Section 8 shows how to parameterize reconstruction/secrecy and Byzantine-fault thresholds separately in the low-threshold regime, and Section 9 reduces stake-weighted committees to small integer weights while preserving those guarantees up to a bounded liveness loss. Appendix A reviews Reed–Solomon decoding algorithms.

2 Preliminaries

Notation and group setup. Let λ denote the security parameter, and let $\mathbb{G}$ be a cyclic group of prime order q, written multiplicatively, with a fixed generator g; scalars are elements of the field $\mathbb{Z}_q$. In our deployment $\mathbb{G}$ is the secp256k1 group used by Bitcoin, and signatures follow the BIP-340 Schnorr format. We assume the discrete logarithm problem is hard in $\mathbb{G}$: any PPT adversary, given g^x for a uniformly random $x \leftarrow\!\!\$\, \mathbb{Z}_q$, outputs x with probability negligible in λ. All hash functions, denoted H with the domain clear from context, are modeled as random oracles and are instantiated with distinct domain separators.

Network and blockchain assumptions. We consider a committee of $n = 3f + 1$ parties, of which up to f may be Byzantine. Later sections generalize this to a stake-weighted committee of total weight $W = 3f+1$ in which the Byzantine parties hold total weight at most f. The network is asynchronous, so messages between honest parties are eventually delivered but may be arbitrarily delayed and reordered. We assume a PKI in which each party holds a signature key, used to sign all of its protocol messages, and an encryption key pair for the multi-recipient scheme below. The parties also have access to a total order broadcast (TOB) service, in practice provided by the underlying blockchain. Any party can post a message on the TOB, and all honest parties eventually observe the posted messages in the same order. Moreover, this message order may not be same as the order in which parties submit

the messages, but every message from the honest party eventually appears on the TOB service. (See state machine replication definition from [MR21, Sec 2.1] for a more formal definition.) Our protocols use the TOB to post certificates of dealings, to agree on the set of certified dealers, and to mark epoch boundaries. Time is divided into epochs. The committee and its stake distribution may change at an epoch boundary, and key material is carried across by resharing (Section 3). We additionally assume a public random beacon, such as the randomness beacon of the underlying blockchain, whose designated outputs are independent uniform PRF keys, or computationally indistinguishable from them, until their first disclosure to the adversary. Section 5 uses a message-associated beacon and a batch-associated beacon, with the message-to-slot assignment fixed before the last fresh disclosure. The nonce distribution protocol of Section 4 uses reliable broadcast, which guarantees that all honest parties receive the same dealer message. Section 7 shows how to replace reliable broadcast with an information dispersal protocol of lower communication cost.

AVSS. We recall the notion of verifiable secret sharing in the asynchronous setting, augmented with the completeness property that our resharing procedure relies on. A (single-dealer) *secret sharing* of a value $s \in \mathbb{Z}_q$ among a set of parties I assigns to each party P_i a share s_i such that any authorized subset of parties can reconstruct s from their shares, while any unauthorized subset learns nothing about s. We work with the $(f+1)$-out-of-n threshold access structure realized by Shamir sharing: the dealer picks a degree-f polynomial p with $\mathsf{p}(0) = s$ and sets $s_i = \mathsf{p}(i)$, so that any $f+1$ shares determine s while any f shares are independent of it. A *verifiable* secret sharing (VSS) additionally lets each receiver check that the shares it obtains are consistent with a unique degree-f polynomial, even when the dealer is malicious; in our Pedersen-style instantiation this is achieved by publishing the commitments $C_k = g^{c_k}$ to the polynomial coefficients and verifying $g^{s_i} = \prod_{k=0}^{f} C_k^{i^k}$.

Multi-recipient encryption. We use the Multi-Encryption Gadget (MEGa) of [GS22] which is a multi-recipient extension of the encryption of [SG98].

- $\mathsf{KeyGen}(1^\lambda) \to (\mathsf{sk}, \mathsf{pk})$: A probabilistic key generation algorithm that takes a security parameter 1^λ, and outputs a secret key, public key tuple $(\mathsf{sk}, \mathsf{pk})$.
- $E \leftarrow \mathsf{Enc}(\mathsf{sid}, PK, (m_\ell)_{\ell \in I})$: encrypts one plaintext m_ℓ under each public key $\mathsf{pk}_\ell \in PK$, producing a single ciphertext E whose ℓ-th component is intended for party P_ℓ. The session identifier sid domain-separates the encryption (in particular, each dealer P_j uses a distinct sid_j).
- $m_i / \perp \leftarrow \mathsf{Dec}(\mathsf{sid}, \mathsf{sk}_i, i, E)$: decrypts the i-th component of E under sk_i.
- $(m, \pi) \leftarrow \mathsf{Reveal}(\mathsf{sid}, \mathsf{sk}_i, i, E)$: produces a plaintext and publicly verifiable proof proving that the i-th component of E decrypts to the given plaintext.
- $0/1 \leftarrow \mathsf{Vf}(\mathsf{sid}, \mathsf{pk}_i, m, \pi, i, E)$: checks that the proof π is valid for the i-th component of E under pk_i, with claimed plaintext m.

See [GS22] for security definitions. In a nutshell, we require that in an Associated-data-only CCA (ADO-CCA) game in which the adversary controls a subset of the parties and can choose their keys after seeing the honest parties' keys, an encryption with sid sid is secure even if the adversary can continually query Reveal with $\mathsf{sid}' \neq \mathsf{sid}$ and with the honest parties' keys.

Reed-Solomon Decoding. Reed-Solomon decoding is an important component of our online signing phase: the partial signatures received by the aggregator are evaluations of a bounded-degree polynomial, so honestly generated partials form a Reed-Solomon codeword and maliciously corrupted partials are simply errors in that codeword (Sections 5 and 8). Rather than identifying faulty contributors via per-partial verification, the aggregator decodes: given enough received partials relative to the number of errors, error correction recovers the underlying polynomial and hence the aggregate signature in a single shot.

Our setting imposes two requirements that narrow the design space. First, the code is defined over the prime scalar field of the signing curve, not the binary extension fields $GF(2^m)$ targeted by most off-the-shelf implementations. Second, the evaluation points are the identities of the responding parties, an arbitrary subset of field elements rather than a structured domain such as a complete set

of roots of unity, although structured domains admit useful optimizations that we note below. With these constraints in mind, we considered several candidate RS decoding algorithms, and went with Gao's extended-Euclidean-based algorithm. Details about these algorithms are given in Appendix A.

3 AVSS with Completeness

We consider the asynchronous setting with $n = 3f + 1$ parties, at most f of which are Byzantine, communicating over point-to-point channels together with a total-order broadcast (TOB) channel. In our protocol several parties act as dealers in parallel, and each honest party plays the role of a receiver for every dealing.

The completeness property is the additional guarantee beyond standard AVSS: it ensures that *all* honest parties (of total weight at least $2f + 1$ in the weighted setting), rather than merely the honest parties of total weight at least $f + 1$ guaranteed among the $2f + 1$ signers of a dealer's certificate, end up holding valid shares. This is what enables the resharing procedure of Section 3.2, whose liveness relies on honest parties of total weight at least $f + 1$ holding valid shares and remaining available as dealers even after parties of total weight up to f retire from the committee.

We first describe a Pedersen style Asynchronous Verifiable Secret Sharing (AVSS) protocol with completeness that works for $n = 3f + 1$. The protocol takes the following inputs:

- Party set I (with $|I| = n = 3f + 1$), and the set of dealers $I_{\text{dealers}} \subseteq I$
- Encryption public keys $PK = \{\mathsf{pk}_j : j \in I\}$
- Set $S = \{s_i : i \in I_{\text{dealers}}\}$ of dealer contributions for each dealer $P_i \in I_{\text{dealers}}$.

The protocol outputs:

- Set $I_{\text{sel}} \subseteq I_{\text{dealers}}$ of the first $f + 1$ dealers, in TOB order, that posted valid aggregate certificates (selected in the completeness phase below)
- Component VSS verification keys $\{C_{j,k}\}_{j \in I_{\text{sel}}, k \in [0,f]}$.
- Component secret shares $\{s_{j,i}\}_{j \in I_{\text{sel}}, i \in I}$.

Session identifiers. Every multi-recipient encryption is keyed by a session identifier that domain-separates ciphertexts across chains, epochs, protocol instances, and dealers. Concretely, a base identifier for a protocol instance is derived as

$$\mathsf{sid}_{\text{base}} \leftarrow H\big(\mathsf{chain_id},\ \mathsf{epoch},\ \mathsf{protocol}\big),$$

where $\mathsf{protocol}$ distinguishes DKG, key resharing, and (for nonces) the batch index, and H is a random oracle with a fixed domain separator. Each dealer P_i then derives a per-dealer identifier

$$\mathsf{sid}_i \leftarrow H_{\mathsf{sid}_{\text{base}}}\big(P_i\big)$$

by hashing the dealer's identity under a random oracle keyed by $\mathsf{sid}_{\text{base}}$. All $\mathsf{Enc}/\mathsf{Dec}/\mathsf{Reveal}/\mathsf{Vf}$ calls for dealer P_i's ciphertext use this sid_i. For key resharing, where each virtual dealer (j, u) re-shares a distinct share, the dealer identity is further bound to the share index u, yielding a distinct $\mathsf{sid}_{(j,u)}$. Batch nonce generation likewise includes the batch index in $\mathsf{sid}_{\text{base}}$ before deriving the per-dealer identifier (Section 4).

AVSS protocol. The AVSS protocol proceeds as follows:

1. Dealer $P_i \in I_{\text{dealers}}$:
   (a) Sample a uniformly random degree-f polynomial $\mathsf{p}_i(X) = c_{i,0} + c_{i,1}X + \cdots + c_{i,f}X^f$ from $\mathbb{Z}_q[X]$, subject to the constraint $c_{i,0} = s_i$.
   (b) Compute $C_{i,k} \leftarrow g^{c_{i,k}}$ for $k \in [0, f]$.
   (c) Compute $s_{i,j} \leftarrow \mathsf{p}_i(j)$ for $j \in I$.

(d) Sample $E_i \leftarrow \mathsf{Enc}(\mathsf{sid}_i, PK, (s_{i,j})_{j\in I})$ where $PK = \{\mathsf{pk}_j : j \in I\}$.

(e) Send to all parties in I the tuple $(C_{i,0}, C_{i,1}, \ldots, C_{i,f}, E_i)$.

(f) Receive $2f+1$ signatures on the message and post aggregated signature σ_i to the TOB.

2. Receiver P_i:

(a) For dealer message received from P_j:

i. Sanity checks - Check the message length, number of encryptions, that all elements belong to the right groups, ZK proofs are valid, etc.

ii. If any of those checks failed then ignore the message and skip. (Note that all honest parties will do the same in this case.)

iii. Decrypt the i-th component of E_j as $s_{j,i} \leftarrow \mathsf{Dec}(\mathsf{sid}_j, \mathsf{sk}_i, i, E_j)$ (may be $\perp$) ,

- Verify that $g^{s_{j,i}} = \prod_{k=0}^{f} C_{j,k}^{i^k}$.
- If valid, sign the hash of the dealer message and send to dealer P_j. Otherwise, ignore the message.

3. Completeness phase P_i:

(a) Let I_{sel} be the set of the first $f+1$ dealers, in TOB order, whose valid aggregate signatures σ_j appear on the TOB.

(b) For each dealer $P_j \in I_{\text{sel}}$:

i. If P_i did not sign in σ_j (i.e., P_i's signature is not included in the aggregate), then:

- Fetch the certificate-bound dealing from its signers if necessary.
- Decrypt $s_{j,i} = \mathsf{Dec}(\mathsf{sid}_j, \mathsf{sk}_i, i, E_j)$.
- If $s_{j,i} = \perp$ (decryption failed) or $g^{s_{j,i}} \neq \prod_{k=0}^{f} C_{j,k}^{i^k}$ (verification failed), then compute $F_{i,j} = \mathsf{Reveal}(\mathsf{sid}_j, \mathsf{sk}_i, i, E_j)$. If $F_{i,j} \neq \perp$, then broadcast $F_{i,j}$.

(c) For confirmation message $F_{j,k} \neq \perp$ received from P_j:

i. If selection of P_k is not yet known, buffer the complaint; if $P_k \notin I_{\text{sel}}$ after selection, discard it.

ii. If needed, fetch P_k's dealer message from the signers of σ_k over P2P and check that it matches the hash bound by σ_k on the TOB. Parse $F_{j,k}$ as $(s_{k,j}, \pi)$. Check if $\mathsf{Vf}(\mathsf{sid}_k, \mathsf{pk}_j, s_{k,j}, \pi, j, E_k)$, and $g^{s_{k,j}} \neq \prod_{\ell=0}^{f} C_{k,\ell}^{j^\ell}$.

iii. If P_i is a signer of σ_k and the check passes, send P_i's already-validated share $s_{k,i}$ to P_j.

(d) Wait for enough responses from other parties to compute the missing shares by interpolation.

(e) Output I_{sel}, $\{C_{j,k}\}_{j\in I_{\text{sel}}, k\in[0,f]}$, and $\{s_{j,i}\}_{j\in I_{\text{sel}}}$.

Note that, for each dealer and session, honest parties sign at most one complete dealing digest, and the first valid certificate on TOB fixes the accepted dealing. Moreover, complaints arriving before its certificate or selection are buffered and reconsidered when that information arrives.

Correctness. The AVSS protocol satisfies the following correctness properties:

- **Agreement:** All honest parties agree on the same set I_{sel} of valid dealers. This follows from the TOB assumption, which ensures that all honest parties see the same sequence of aggregate certificates σ_i posted by dealers.
- **Validity:** For each dealer $P_j \in I_{\text{sel}}$ and each honest party $P_i \in I$, the component share $s_{j,i}$ satisfies the verification equation $g^{s_{j,i}} = \prod_{k=0}^{f} C_{j,k}^{i^k}$ with respect to the component VSS verification keys $\{C_{j,k}\}_{k=0}^{f}$. This is guaranteed by the receiver verification step and the completeness phase.
- **Consistency:** For each dealer $P_j \in I_{\text{sel}}$, the component shares $\{s_{j,i}\}_{i\in I}$ form a valid secret sharing of s_j (i.e., they lie on a degree-f polynomial p_j with $\mathsf{p}_j(0) = s_j$). This follows from the dealer's polynomial commitment and the verification checks.
- **Honest dealer inclusion:** Since there are at most f dishonest parties and I_{sel} contains the first $f+1$ dealers who posted aggregate certificates, at least one honest dealer is included in I_{sel}.

Need for completeness. We assume that between consecutive key rotations parties of total weight at most f retire from the committee and are replaced by new parties of total weight at most f. Since Hashi lets only a party that serves in both committees act as a dealer (Section 3.2), resharing requires honest parties of total weight at least $f+1$ in the intersection of the old and the new committee to hold valid shares. It follows that honest parties of total weight at least $2f+1$ in the old committee must hold valid shares, so that even after weight f of them retires, the remaining honest weight $f+1$ can reshare the key. This is the guarantee that completeness adds beyond standard AVSS, which only ensures valid shares for the honest parties that contributed to the dealer certificates.

3.1 DKG

The distributed key generation protocol instantiates the AVSS template (Section 3) with *physical* dealers and *virtual* receivers. We represent *weights* by expanding each physical party P_j of weight w_j into w_j virtual receiver indices $(j,1),\ldots,(j,w_j)$, each associated with a distinct canonical evaluation point $x_{(j,u)} \in \mathbb{Z}_q^*$ determined by a fixed injective map. Let $I^\star = \{(j,u) : j \in I, u \in [1,w_j]\}$ and $W = \sum_{j\in I} w_j$, and assume $W = 3f+1$.

The virtual indices do not correspond to independent machines, dealers, or encryption keys; they are meant as "bookkeeping" to allow the weighted committee to be modeled as an ordinary Shamir sharing over W evaluation points. A physical party P_j owns and stores all shares corresponding to its virtual indices (j,u), and all such indices use P_j's physical encryption key. Consequently, a coalition of physical parties of total weight t controls exactly t virtual shares, so weighted reconstruction reduces to standard Shamir reconstruction over the virtual evaluation points.

How physical-dealer DKG differs from virtual-dealer DKG. Each physical party contributes *one* degree-f polynomial p_j (one constant term s_j). That polynomial is evaluated at *every* virtual point $x_{(i,v)}$, so a heavy party does not act as w_j independent dealers. Weight enters on the receiving side instead: each dealing produces one share per virtual point, and a party of weight w_i collects the w_i shares at its own points from every dealing. Algebraically, if I_{sel} is the set of selected physical dealers, define

$$\mathbb{P}(X) := \sum_{j\in I_{\mathrm{sel}}} \mathsf{p}_j(X)\,.$$

Then $\mathbb{P}$ has degree at most f, satisfies $\mathbb{P}(0) = \sum_{j\in I_{\mathrm{sel}}} s_j$, and $\mathbb{P}(x_{(i,v)}) = \sum_{j\in I_{\mathrm{sel}}} \mathsf{p}_j(x_{(i,v)})$. Thus the final Shamir object is still a single degree-f polynomial over the virtual evaluation points, and the joint key is $\mathsf{vk} = g^{\mathbb{P}(0)} = \prod_{j\in I_{\mathrm{sel}}} C_{j,0}$.

Concretely, we run AVSS with the following parameters:

- **Receiver / virtual party set:** $I^\star$ (total weight $W = 3f+1$).
- **Dealer set:** $I_{\mathrm{dealers}} = I$ (each physical party P_j is one dealer).
- **Encryption keys:** $PK = \{\mathsf{pk}_j : j \in I\}$; ciphertext component for virtual party (i,v) is encrypted under pk_i.
- **Dealer inputs:** each P_j samples $s_j \leftarrow\!\$\, \mathbb{Z}_q$ and uses p_j with $\mathsf{p}_j(0) = s_j$.
- **Per-dealer shares:** for each virtual party $(i,v) \in I^\star$, dealer P_j sets $s_{j,(i,v)} := \mathsf{p}_j(x_{(i,v)})$ (in the generic AVSS notation, the receiver index is the virtual identity (i,v) rather than i alone).
- **Receiver verification:** $g^{s_{j,(i,v)}} = \prod_{k=0}^{f} C_{j,k}^{x_{(i,v)}^k}$.
- **Dealer selection:** Let $I_{\mathrm{sel}} \subseteq I$ be the set of the first valid *physical* dealers, in TOB order, whose total weight $\sum_{j\in I_{\mathrm{sel}}} w_j$ reaches $f+1$.
- **Outputs to each virtual party** (i,v)**:**
  - Component VSS keys $\{C_{j,k}\}_{j\in I_{\mathrm{sel}},\, k\in[0,f]}$ and component shares $\{s_{j,(i,v)}\}_{j\in I_{\mathrm{sel}}}$ with $s_{j,(i,v)} = \mathsf{p}_j(x_{(i,v)})$.
- **DKG finalization:** Each virtual party (i,v) computes

- Joint verification key $\mathsf{vk} = \prod_{j \in I_{\text{sel}}} C_{j,0}$;
- Aggregated VSS verification keys $\{\prod_{j \in I_{\text{sel}}} C_{j,k}\}_{k=0}^{f}$;
- Final secret share $s_{(i,v)} = \sum_{j \in I_{\text{sel}}} s_{j,(i,v)} = \mathbb{P}(x_{(i,v)})$.

The master secret is $s = \mathbb{P}(0) = \sum_{j \in I_{\text{sel}}} s_j$, which remains unknown to individual parties. The threshold property is unchanged in the virtual-point model: any set of virtual parties whose total weight is at least $f + 1$ can reconstruct degree-f secrets / signatures, while any coalition of physical parties of total weight at most f is bounded by the usual AVSS soundness.

The completeness property of the AVSS protocol ensures that all honest parties whose total weight is at least $2f + 1$ will eventually obtain valid shares for vk. This is critical for supporting subsequent key resharing operations, as described in the next subsection.

3.2 Key Resharing

The key resharing protocol allows the system to maintain the same master secret while rotating the party set. Between resharing operations, we assume that the total retiring weight is at most f and is replaced by new parties of total weight at most f, ensuring that the remaining parties from the previous configuration have total weight at least $2f + 1$. We again use the canonical evaluation points $x_{(j,u)}$ derived from the pair (j, u).

Let I_{old} denote the set of parties from the previous DKG or resharing operation, and let I_{new} denote the set of parties in the new configuration. Define the virtual sets $I^{\star}_{\text{old}} = \{(j, u) : j \in I_{\text{old}}, u \in [1, w_j]\}$ and $I^{\star}_{\text{new}} = \{(j, u) : j \in I_{\text{new}}, u \in [1, w_j]\}$. The protocol requires only that virtual dealers of total weight at least $f + 1$ hold valid shares of the old key and take part in the resharing; membership in the new committee is not needed, and a retiring party can serve as a dealer. Although not strictly necessary, Hashi restricts dealers to the parties that serve in both committees, since a party that leaves the committee stops running the protocol at the epoch boundary. We therefore require the total weight of $I_{\text{old}} \cap I_{\text{new}}$ to be at least $2f + 1$, which together with the retirement bound above leaves honest dealers of total weight at least $f + 1$. Each party $P_j \in I_{\text{old}}$ has valid shares $\{s_{(j,u)}\}_{u \in [1,w_j]}$ from the previous configuration and the corresponding partial verification keys $\{C_{(j,u),0} = g^{s_{(j,u)}}\}_{u \in [1,w_j]}$ from the previous AVSS run.

Why resharing uses virtual dealers. DKG used one polynomial per *physical* party, but the object each virtual slot holds afterward is a *different* scalar $s_{(j,u)} = \mathbb{P}(x_{(j,u)})$. A single new polynomial from P_j has only one constant term, so it cannot simultaneously re-share w_j unrelated secrets $s_{(j,1)}, \ldots, s_{(j,w_j)}$ unless they lie on one degree-f polynomial in the $x_{(j,u)}$, which they need not. Therefore resharing is done with *virtual* dealers (j, u) from the old committee $I^{\star}_{\text{old}}$, each re-sharing its own $s_{(j,u)}$ as in the AVSS template (constant term $C_{(j,u),0} = g^{s_{(j,u)}}$), while receivers remain virtual parties in $I^{\star}_{\text{new}}$ with points $x_{(i,v)}$ under the new weights.

The resharing protocol is obtained by running the AVSS protocol (Section 3) with the following parameters:

- **AVSS Inputs:**
  - Virtual party set $I^{\star}_{\text{new}}$ (the new virtual party set)
  - Virtual dealer set $I^{\star}_{\text{dealers}} = \{(j, u) : j \in I_{\text{old}},\ u \in [1, w_{j,\text{old}}]\}$ (old-committee virtual indices under *old* weights act as dealers; in Hashi these are restricted to the parties that remain in the new committee, as noted above)
  - Encryption public keys $PK_{\text{new}} = \{\mathsf{pk}_j : j \in I_{\text{new}}\}$, where each party in I_{new} generates a fresh encryption key pair for the new configuration (encryption keys are not reused across configurations)
  - Dealer contributions $S = \{s_{(j,u)} : (j, u) \in I^{\star}_{\text{dealers}}\}$ where each $s_{(j,u)}$ is an existing share from the previous configuration
- **Additional receiver checks:** Each virtual party in $I^{\star}_{\text{new}}$ verifies that $C_{(j,u),0}$ matches the expected partial verification key $g^{s_{(j,u)}}$ from the previous configuration for each $(j, u) \in I^{\star}_{\text{dealers}}$.
- **AVSS Outputs:** Each virtual party $(i, v) \in I^{\star}_{\text{new}}$ receives from AVSS:

- Set $I^\star_{\mathrm{sel}} \subseteq I^\star_{\mathrm{dealers}}$ of the first valid virtual dealers whose total weight reaches $f+1$
- Component VSS verification keys $\{C_{(j,u),k}\}_{(j,u)\in I^\star_{\mathrm{sel}},k\in[0,f]}$ (where $C_{(j,u),k} = g^{c_{(j,u),k}}$ for dealer (j,u)'s polynomial coefficients, with $C_{(j,u),0} = g^{s_{(j,u)}}$)
- Component secret shares $\{s_{(j,u),(i,v)}\}_{(j,u)\in I^\star_{\mathrm{sel}}}$ (share $s_{(j,u),(i,v)} = \mathsf{p}_{(j,u)}(x_{(i,v)})$ from virtual dealer (j,u))

- **Resharing Finalization:** Each virtual party $(i,v) \in I^\star_{\mathrm{new}}$ computes:
  - Verification that $\mathsf{vk}' = \prod_{(j,u)\in I^\star_{\mathrm{sel}}} C_{(j,u),0}^{\lambda_{0,I^\star_{\mathrm{sel}},(j,u)}} = \mathsf{vk}$ (key continuity check), where $\lambda_{0,I^\star_{\mathrm{sel}},(j,u)}$ are the Lagrange coefficients for interpolating at point 0 using evaluation points $\{x_{(j,u)}\}_{(j,u)\in I^\star_{\mathrm{sel}}}$
  - Aggregated VSS verification keys $\{\prod_{(j,u)\in I^\star_{\mathrm{sel}}} C_{(j,u),k}^{\lambda_{0,I^\star_{\mathrm{sel}},(j,u)}}\}_{k=0}^{f}$
  - New secret share $s'_{(i,v)} = \sum_{(j,u)\in I^\star_{\mathrm{sel}}} \lambda_{0,I^\star_{\mathrm{sel}},(j,u)} \cdot s_{(j,u),(i,v)}$

The new shared secret is implicitly defined as $s' = \sum_{(j,u)\in I^\star_{\mathrm{sel}}} \lambda_{0,I^\star_{\mathrm{sel}},(j,u)} \cdot s_{(j,u)}$, which by the properties of Lagrange interpolation equals the original master secret s from the previous configuration. The verification key remains unchanged: $\mathsf{vk}' = \prod_{(j,u)\in I^\star_{\mathrm{sel}}} C_{(j,u),0}^{\lambda_{0,I^\star_{\mathrm{sel}},(j,u)}} = g^{s'} = g^s = \mathsf{vk}$. This ensures key continuity, so that signatures generated with the new shares are indistinguishable from signatures generated with the original shares, and the public verification key need not be updated in any external systems.

For clarity, the above description uses unchanged f; if the new bound is f', retain $f+1$ old dealers but use degree f', $2f'+1$ recipient certificates, and $f'+1$ recovery responses in the new committee. Old and new virtual indices use their respective epoch weights.

The completeness property of the underlying AVSS protocol ensures that all honest parties in I_{new} with total weight at least $2f+1$ will obtain valid shares for the refreshed key, enabling subsequent resharing operations to proceed smoothly even if total weight f rotates out before the next resharing.

At the transition, old access is revoked and parties register fresh encryption and authentication keys for their new roles. Before a new-role corruption exposes state, honest old roles erase their key shares, nonce state, dealer polynomials, random coins, and old encryption and authentication secret keys; old credentials are no longer accepted. Thus later corruption reveals only the current role's state, as assumed in Section 6.

4 Batch AVSS

Every Schnorr signature produced by the protocol of Section 5 consumes a fresh secret-shared nonce, whose combined secret must remain hidden from every coalition of weight at most f while its public commitment enters the Fiat–Shamir challenge. Since nonce generation is independent of the messages to be signed, we generate nonces offline in large batches rather than one sharing at a time. Each committee member acts as a dealer and verifiably secret-shares many nonce polynomials at once, which amortizes the broadcast and verification costs of the batch. The protocol below is a batch variant of the AVSS of Section 3. The dealer commits to the constant term of each nonce polynomial and encrypts all shares in a single multi-recipient ciphertext. The dealer also publishes a masking polynomial that folds the entire batch together under random-oracle challenges, and recipients verify the consistency of all sharings by checking a single polynomial identity. Certified batches from different dealers are later combined through a super-invertible matrix into presigning slots (Section 5). We describe the protocol for a single dealer in the weighted model of Section 3, where each physical party is expanded into virtual receivers at canonical evaluation points.

Let L denote the *batch size per unit of weight*: a fixed system parameter, identical for all dealers and known to all parties, chosen to amortize the broadcast and verification costs over the batch. A dealer of weight w_D deals $w_D L$ nonce polynomials in a single batch, so the supply of nonces a dealer contributes is proportional to its stake.

We have a dealer D with weight w_D who wants to distribute $w_D L$ nonces to virtual parties in $I^\star = \{(i,v) : i \in I, v \in [1,w_i]\}$ with total weight $W = 3f+1$. Each virtual party (i,v) has canonical evaluation point $x_{(i,v)}$. Let sid be the per-dealer session identifier for D, derived as in Section 3. The dealer does the following:

1. Sample $w_D L$ uniformly random degree-f polynomials $\mathsf{p}_l(X) = c_{l,0} + c_{l,1}X + \cdots + c_{l,f}X^f$ from $\mathbb{Z}_q[X]$ for $l \in [1, w_D L]$.
2. Sample a uniformly random degree-f polynomial $\mathsf{p}'(X) = c'_0 + c'_1 X + \cdots + c'_f X^f$ from $\mathbb{Z}_q[X]$.
3. For each $l \in [1, w_D L]$, compute $C_l \leftarrow g^{c_{l,0}}$.
4. Compute $C' \leftarrow g^{c'_0}$.
5. Compute $r_{l,(i,v)} \leftarrow \mathsf{p}_l(x_{(i,v)})$ for $(i,v) \in I^\star$ and $l \in [1, w_D L]$.
6. Compute $r'_{(i,v)} \leftarrow \mathsf{p}'(x_{(i,v)})$ for $(i,v) \in I^\star$.
7. Create the multi recipient encryption $E \leftarrow \mathsf{Enc}(\mathsf{sid}, PK^\star, \langle R_{(i,v)} \rangle_{(i,v)\in I^\star})$ where $PK^\star = \{\mathsf{pk}_i : i \in I\}$ (each (i,v) uses pk_i) and $R_{(i,v)} = (r_{1,(i,v)}, \ldots, r_{w_D L,(i,v)}, r'_{(i,v)})$ for $(i,v) \in I^\star$.
8. Compute $\gamma_l \leftarrow \mathsf{H}(\mathsf{sid}, l, \langle C_l \rangle_{l\in[1,w_D L]}, C', E)$ for each $l \in [1, w_D L]$.
9. Compute the polynomial $\mathsf{p}''(X) = \mathsf{p}'(X) + \sum_{l\in[1,w_D L]} \gamma_l \mathsf{p}_l(X)$.
10. Reliable broadcast $(\langle C_l \rangle_{l\in[1,w_D L]}, C', E, \mathsf{p}''(X))$.[1]
11. Wait for signatures from virtual parties of total weight $2f+1$ on the hash of the above message and form a certificate. The reliable broadcast, here, makes sure that every honest party has received the message.
12. Post the certificate on the TOB.

Virtual party $(i,v) \in I^\star$ does the following:

1. For a message received from D do:
   (a) Sanity checks - Check the message length, number of encryptions, that all elements belong to the right groups, that ZK proofs are valid, and that p'' is of the right degree.
   (b) For each $l \in [1, w_D L]$, compute $\gamma_l \leftarrow \mathsf{H}(\mathsf{sid}, l, \langle C_l \rangle_{l\in[1,w_D L]}, C', E)$.
   (c) Verify that $g^{\mathsf{p}''(0)} = C' \cdot \prod_{l\in[1,w_D L]} C_l^{\gamma_l}$.
   (d) Decrypt the (i,v)-th component of E using $\mathsf{Dec}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$. Let $\langle r_{l,(i,v)} \rangle_{l\in[1,w_D L]}, r'_{(i,v)}$ be the decrypted shares.
   (e) Verify that $\mathsf{p}''(x_{(i,v)}) = r'_{(i,v)} + \sum_{l\in[1,w_D L]} \gamma_l r_{l,(i,v)}$.
   (f) If valid, store the decrypted shares and sign the message and send to dealer D. Else ignore the message.
2. For a valid certificate received from D do:
   (a) If needed, fetch D's message from the certificate signers and verify its certified digest.
   (b) If has not stored decrypted shares for that message,
      i. Decrypt the (i,v)-th component of E using $\mathsf{Dec}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$ and check as specified above.
      ii. If all checks succeeded, store the decrypted shares.
      iii. Else, if any of the checks failed, set $F_{(i,v)} = \mathsf{Reveal}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$ and broadcast $F_{(i,v)}$.
      iv. Wait for valid responses from virtual parties of total weight $f+1$ with shares (see below) and interpolate them to recover the shares of (i,v). (In practice we should retry sending $F_{(i,v)}$ every 10 seconds, to make sure honest parties receive the certificate and can then respond.)
3. For each fraud complaint $F_{(j,u)}$ received do:

[1] AVID reduces the leading honest-dealer dispersal cost for this $O(w_D L \cdot W)$ message; control traffic and Byzantine repair are accounted for separately in Section 7.

(a) If no aggregate certificate has been received from D, then ignore.

(b) If the complaint is invalid, then ignore.

(c) Else, set $F_{(i,v)} \leftarrow \mathsf{Reveal}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$.

(d) Send $F_{(i,v)}$ to (j,u).

While for ease of exposition, we write the above code for virtual party $(i,v) \in I^\star$, it trivially works by combining the process for the physical party. In implementation, we implement the virtual-party description per physical party: it verifies all its virtual shares and signs once with its full weight only if every check passes.

Replacing reliable broadcast with AVID. The dealer above disperses the message $(\langle C_l \rangle_{l \in [1, w_D L]}, C', E, \mathsf{p}''(X))$ via reliable broadcast, whose dominant term is the encryption E of size $O(w_D L \cdot W \cdot \lambda)$. Reliably broadcasting a message of this size incurs a communication cost that is a factor of n larger than the message itself. We can avoid this blow-up by instead using the asynchronous verifiable information dispersal (AVID) protocol of Section 7. That protocol disperses the same n-element vector $E = (E_1, \ldots, E_n)$ so that each honest virtual party retrieves its own component E_i, while placing a short proof of correct dispersal on the TOB certifying that at least $f+1$ honest parties received their components; a party that does not obtain a correct component aborts in an identifiable manner. The leading honest-dealer dispersal cost is $O(w_D L \cdot W \cdot \lambda)$, with common-message and authentication traffic added separately. The TOB certificate subsumes the separate certification steps. We refer the reader to Section 7 for the full construction, its integration with the batch AVSS flow, and the safety and liveness analyses.

5 Signature Generation

The DKG of Section 3 establishes the joint key, and the batch AVSS of Section 4 provides certified nonce sharings; this section describes how signatures are produced from them. Certified dealings cannot be used as signing nonces directly, since dealers of total weight up to f may be Byzantine, and a Byzantine dealer knows the secrets of its own dealings and may have chosen them adaptively. In the pre-signing phase, the certified dealings are therefore combined through a super-invertible matrix. From blocks contributed by certified dealers of total weight W_J, of which at least weight $W_J - f$ are honestly generated, the combination produces $M = W_J - f$ nonces per block column, each uniformly random from the adversary's view. Pre-signing outputs a public matrix of nonce commitments and, for each virtual party, a secret matrix of the corresponding shares; each matrix coordinate serves as a presigning slot.

Online signing takes a single round, where a signing instance is assigned a fresh slot. Here, we consider two approaches. In Section 5.1, the nonce of the slot is offset by values derived from the message and batch beacons, and each virtual party broadcasts one Schnorr partial signature.

The aggregator interpolates partial signatures of total weight $f+1$, and uses Reed–Solomon decoding to correct faulty partials when plain interpolation fails. Our analysis in Section 5.1 also includes epoch boundaries and non-hardened key derivation from the joint key.

5.1 Random Beacon-based Solution

We assume all honest virtual parties $(i,v) \in I^\star$ have shares of the secret key, denoted by $s_{(i,v)}$, at evaluation points $x_{(i,v)}$.

Pre-signing. Each party P_i does the following:

1. Act as a dealer in the nonce distribution protocol from Section 4 with weight w_i and L nonces (i.e., distribute $w_i L$ nonces).

2. Waits for certificates on TOB from parties whose total weight reaches $2f+1$. Then, wait another Δ time defined by the network (e.g., 10 checkpoints on Sui) to collect more valid certificates from different parties.

3. Let J be the list of indices of the parties that contributed a certificate in the previous step (either as part of the first total weight $2f+1$ or the ones received during Δ). Note that thanks to ordering on TOB and assuming existence of checkpoints there, the list J is the same for each party.

4. Finalize PresigDealerSet(epoch η, batch index bid, J) on TOB, fixing the ordered dealings and hence D. Fix any pending message-to-slot assignments relying on this batch beacon before requesting β_p; its first disclosure must follow these choices.

5. Let $W_J = \sum_{j \in J} w_j$ and $M = W_J - f$. Prepare $M \cdot L$ presigs:

   (a) Let $J^\star = \{(j,u) : j \in J, u \in [1, w_j]\}$ be the expanded dealer index set, with $|J^\star| = W_J$.

   (b) Let $r_{(j,u),l,(i,v)}, C_{(j,u),l}$ be the l-th share and public key received above from dealer $j \in J$ for virtual party (i,v), corresponding to dealer j's u-th block of L nonces.

   (c) Construct the matrices $\rho_{(i,v)} \leftarrow (r_{(j,u),l,(i,v)})_{(j,u)\in J^\star, l\in[1,L]}$ and $C \leftarrow (C_{(j,u),l})_{(j,u)\in J^\star, l\in[1,L]}$.

   (d) Construct the SI matrix $S \in \mathbb{F}^{M \times W_J}$ from the ordered index set $J^\star$.

   (e) Compute $T_{(i,v)} \leftarrow S \cdot \rho_{(i,v)}$, and $D \leftarrow S \cdot C$.

   (f) Store $T_{(i,v)} \in \mathbb{F}^{M \times L}$ and $D \in \mathbb{G}^{M \times L}$ for later use. These are matrices of shares of nonces and their corresponding public commitments. The $T_{(i,v)}$ matrix is secret to virtual party (i,v) and D is public.

Signature Generation. Each virtual party $(i,v) \in I^\star$ does the following to get a signature for message m_k from a withdrawal set ω_k; there will be one or more messages per withdrawal. We expect random beacon value β_k from TOB associated with the withdrawal:

1. Wait for the assigned presign and designated beacon values, then compute $\delta_k = PRF_{\beta_k}(m_k)$ and $\delta_p = PRF_{\beta_p}(\eta, \mathsf{bid}, J, t, l, D_{t,l})$, where $t \in [1, M]$ and $l \in [1, L]$.

2. Set $R_k \leftarrow (D)_{t,l} \cdot g^{\delta_p + \delta_k}$

3. Compute $\sigma_{(i,v)} \leftarrow (T_{(i,v)})_{t,l} + \mathsf{H}(m_k, R_k, \mathsf{vk}) \cdot s_{(i,v)}$.

4. Broadcast $\sigma_{(i,v)}$.

Signature Aggregation. The signature aggregation is done as follows for message m_k:

1. Get beacon values β_k and β_p for the message and the pre-sign batch respectively. Note that they will be already available on TOB at this stage. Compute δ_p and δ_k using PRF on β_p and β_k as above.

2. Set $R_k \leftarrow (D)_{t,l} \cdot g^{\delta_k + \delta_p}$

3. Wait for partial signatures from virtual parties of total weight $f+1$, interpolate them using evaluation points $\{x_{(i,v)}\}$ to get σ'_k and check if the signature $(R_k, \sigma'_k + \delta_p + \delta_k)$ verifies for message m_k. If so, output the signature.

4. Else, retry as further authenticated partials arrive, counting each virtual evaluation point at most once (at most W points).

   (a) On receiving $n' > f+1$ signatures from virtual parties, Reed Solomon (RS) decode, correcting up to $\lfloor (n' - f - 1)/2 \rfloor$ errors. If decoding fails, then continue loop. Refer to Section A for the RS decoding protocol.

   (b) If decoding succeeds and outputs σ'_k, then check the signature as before and output it if valid.

5. If all available decoding attempts fail, keep waiting for additional validly identified partials; output a terminal failure only if the epoch is irrevocably retired.

Note that in the asynchronous communication setting the virtual parties might not finish the signing protocol across an epoch change: the protocol above assumes that honest virtual parties of total weight at least $2f+1$ hold shares, whereas only honest virtual parties of total weight at least $f+1$ are assumed to continue into the next committee. Thus, we require that ongoing signing protocols are aborted by the end of the epoch, and restarted with the new committee in the next epoch, with new nonces, etc.

5.2 Extending the Protocol

Epoch Boundaries. Aborting at epoch boundaries and restarting is safe; liveness additionally requires an epoch long enough for the request to finish. Safety holds because all pre-signing messages are scoped to an epoch through the session identifier sid, which includes the epoch id, and each signing instance consumes a unique matrix coordinate (t, l). Since nonces are freshly generated each epoch, partial signatures from different epochs are evaluations of different polynomials and cannot be combined into a valid aggregate. Partial signatures left over from an aborted epoch reveal no secret-key information. At worst, the adversary can combine the leftover partial signatures of an instance with the shares it already holds and complete a signature on that instance's message, which was authorized for signing before the abort. The completed signature uses its own nonce, so this is harmless, and leftover partials cannot yield a signature on any other message.

Upon restart in the new epoch, parties sample fresh nonces and use a fresh beacon value β_k for message, so no presigning slot is ever consumed under two different challenges. Liveness follows because once the new committee contains honest virtual parties of total weight at least $2f+1$ with valid shares, the pre-signing phase can be rerun and the signing protocol will complete. Any messages that were in flight at the boundary can be re-issued under the new sid and will finalize with the new committee. The TOB boundary gives a clean cut between epochs, so the worst-case effect of an abort is a one-epoch delay with no impact on safety.

Key Derivation. A common requirement in blockchain applications is the ability to derive multiple public keys from a single master key, enabling features like hierarchical deterministic wallets and address rotation without re-running the DKG protocol. Following the approach used in systems like the Internet Computer [DFI24], we support non-hardened key derivation as follows.

Let vk denote the master MPC public key (output of DKG) and let s denote its corresponding secret key, shared among the virtual parties. For a derivation tag $\mathsf{tag} \in \{0,1\}^*$ (which can be a counter, a path, or any application-specific identifier), the derived public key is:

$$\mathsf{vk}_{\mathsf{tag}} = \mathsf{vk} \cdot g^{\mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})}$$

where $\mathsf{H}_{\mathsf{field}} : \{0,1\}^* \to \mathbb{Z}_q$ is a hash function modeled as a random oracle. The corresponding derived secret key is implicitly $s_{\mathsf{tag}} = s + \mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})$, though no party ever holds s_{tag} in the clear.

To sign a message m under the derived key $\mathsf{vk}_{\mathsf{tag}}$, virtual parties first compute a partial signature as usual:

$$\sigma'_{(i,v)} = (T_{(i,v)})_{t,l} + \mathsf{H}(m, R, \mathsf{vk}_{\mathsf{tag}}) \cdot s_{(i,v)}$$

After aggregation yields σ', the final signature is adjusted by adding the public derivation offset:

$$\sigma = \sigma' + \delta_p + \delta_k + \mathsf{H}(m, R, \mathsf{vk}_{\mathsf{tag}}) \cdot \mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})$$

This produces a valid Schnorr signature (R, σ) under the derived key $\mathsf{vk}_{\mathsf{tag}}$, since:

$$g^{\sigma} = g^{r+\delta_p+\delta_k+\mathsf{H}(m,R,\mathsf{vk}_{\mathsf{tag}})\cdot s+\mathsf{H}(m,R,\mathsf{vk}_{\mathsf{tag}})\cdot\mathsf{H}_{\mathsf{field}}(\mathsf{vk}\|\mathsf{tag})} = R \cdot \mathsf{vk}_{\mathsf{tag}}^{\mathsf{H}(m,R,\mathsf{vk}_{\mathsf{tag}})}$$

Note that the derivation offset $\mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})$ is publicly computable from vk and tag, so the adjustment can be performed by any party (or even an external aggregator) without access to secret shares. This enables efficient key derivation without additional MPC rounds.

6 Security of the MPC Signature Protocol

We formalize the unforgeability of the MPC signing protocol in the random-oracle model against *epoch-wise static*, rushing adversaries [OY91, HJKY95] corrupting total weight up to $f = \lfloor (W-1)/3 \rfloor$, where $W = \sum_{j=1}^{n} w_j$ is the total weight. The corrupted set is fixed at the beginning of each epoch and can only change during resharing, where the new corrupted set is declared as part of the resharing query. Each epoch uses fresh encryption keys, honest parties erase their pre-resharing state once resharing completes, and a party corrupted in a later epoch reveals only its current post-resharing state.

Our multi-epoch claim uses this as an explicit separation of old and new roles: old corruption exposes only old-role state, and new corruption exposes only new-role state. A cured party generates fresh encryption and authentication keys after old access is revoked; an honest old role erases its key shares, nonce state, dealer polynomials, and random coins before a newly corrupted role exposes state.

For Section 5.1, the assignment "corresponding to k" must fix $(m_k, \mathsf{vk}, \eta, \mathsf{bid}, J, t, l, D_{t,l})$ before the last fresh beacon for that request is disclosed to the adversary (use the effective key for key derivation). The TOB record $\mathsf{PresigDealerSet}(\eta, \mathsf{bid}, J)$ fixes the ordered J and uniquely certified dealings; its cutoff is determined by common TOB positions, and J is finalized once. Each raw nonce dealing belongs to one batch, and each slot is reserved once across all messages and keys, including aborted requests. All requests relying on one disclosure as their last fresh beacon are fixed before that disclosure. Earlier batch beacons may be public when later requests use a fresh message beacon. The two beacon roles use distinct, predetermined rounds, with independent uniform PRF keys, or a realization computationally indistinguishable from that interface. Throughout, we retain the shorthand

$$\delta := \delta_k + \delta_p \pmod q, \qquad \delta_k = \mathsf{PRF}_{\beta_k}(m_k), \qquad \delta_p = \mathsf{PRF}_{\beta_p}(\eta, \mathsf{bid}, J, t, l, D_{t,l}).$$

The PRF has range $\mathbb{Z}_q$; distinct inputs do not guarantee distinct outputs. All encodings and oracle domains are unambiguous and separated, including those used by encryption. No nonce or signing instance begins before the epoch's key-sharing state is finalized.

Definition 1 (mEUF-CMA for MPC Signing)**.** *Let Π be the MPC signing protocol, which on input a party set I with weights $\{w_j\}$ (total weight W) outputs verification keys $\{\mathsf{vk}_{(j,u)}\}_{(j,u)\in I^\star}$ for each virtual index and an implicit joint key vk, where $I^\star = \{(j,u) : j \in I,\ u \in [1, w_j]\}$ is the virtual index set.*

Consider the experiment $\mathbf{Exp}_{\mathcal{A}}^{mEUF}(\lambda)$:

1. ***Setup**: Run the DKG/initialization of Π with a corrupted set $A \subseteq I$ of total weight at most f, producing $\{\mathsf{vk}_{(j,u)}\}_{(j,u)\in I^\star}$ and defining the joint key vk. Hand to $\mathcal{A}$ the internal states of corrupted parties and the public transcript. In this and in every protocol execution run by the oracles below, $\mathcal{A}$ chooses the messages of the currently corrupted parties and may send them after seeing the honest parties' messages (rushing). Initialize epoch counter $e \leftarrow 0$.*

2. ***Signing oracle**: $\mathcal{A}$ may adaptively query an oracle $\mathcal{O}_{\mathsf{Sign}}(m)$ that initiates a fresh signing instance of Π on m under the current epoch state. Instances and preprocessing may run concurrently; $\mathcal{A}$ sees honest partials as delivered and may delay completion. The oracle returns the signature when the instance completes, using fresh signing material for every query. The message is recorded as queried upon authorization, even if the instance remains pending or is later retired.*[2]

3. ***Resharing oracle**: $\mathcal{A}$ may adaptively query an oracle $\mathcal{O}_{\mathsf{reshare}}(I', \{w'_j\}, A')$ where I' is a new party set with weights $\{w'_j\}$ (total weight W', with $f' = \lfloor (W'-1)/3 \rfloor$) and $A' \subseteq I'$ is the set of corrupted parties of total weight at most f'. The oracle executes the key resharing protocol from the current committee to the new committee, producing new shares of the same joint secret s (so that vk remains unchanged). The oracle hands to $\mathcal{A}$ the internal states of corrupted parties in both committees and the public transcript; a party corrupted only in the new committee reveals only its post-resharing state. Update epoch $e \leftarrow e+1$ and replace $I, \{w_j\}, A$ with $I', \{w'_j\}, A'$.*

[2] For our construction, each query is assigned a unique presigning coordinate (sid, t, l), where (t, l) indexes one entry of the presigning matrices produced for session sid, and σ is produced by executing the protocol's nonce-generation, presigning, and online signing on that coordinate. Repeated queries on the same coordinate are disallowed; the experiment enforces one-time use per coordinate.

4. ***Output***: *Eventually* $\mathcal{A}$ *outputs* (m^*, σ^*). *The experiment returns 1 iff* $\mathsf{Vf}(\sigma^*, m^*, \mathsf{vk}) = 1$ *and* m^* *was not queried to* $\mathcal{O}_{\mathsf{Sign}}$.

We say Π *is mEUF-CMA secure if* $\Pr\left[\mathbf{Exp}^{mEUF}_{\mathcal{A}}(\lambda) = 1\right]$ *is negligible for all PPT* $\mathcal{A}$.

Theorem 1 (Main)**.** *The MPC signature protocol* Π *is mEUF-CMA secure per Definition 1 in the random-oracle model, assuming the discrete logarithm problem is hard in* $\mathbb{G}$*, the multi-recipient encryption scheme is ADO-CCA secure [GS22], and the signature scheme used to certify dealings is unforgeable.*

Here we also assume the stated beacon/transition interfaces and PRF security, and require the statistical errors bounded below, for unforgeability, to be negligible for the chosen parameters. The folded-check error $\varepsilon_{\text{fold}}$ *is needed only for liveness. ADO-CCA means associated-data-only CCA; the chosen multi-recipient scheme must support multiple challenge ciphertexts and repeated public keys at different virtual recipient indices, with recipient/session binding and sound, uniquely bound reveal proofs. Certificate digests bind the complete checked dealing. These are explicit requirements on the instantiated primitives.*

Proof. Assume an adversary $\mathcal{A}$ wins the experiment with non-negligible probability. The simulator described below emulates the protocol transcript, answers signing queries straight-line by programming the random oracle, and, upon a successful forgery, applies a forking argument to obtain two accepting transcripts with the same R and distinct challenges. From these it extracts $\log_g(\mathsf{vk})$ and, using the relation $\mathsf{vk} = (\mathsf{vk}^*)^\tau \cdot g^w$ with known w and invertible τ, recovers the challenger's discrete logarithm x^*. This contradicts the hardness of discrete logarithm. We then show that the simulator is indistinguishable from the real execution. □

6.1 Detailed Simulation

Assume towards contradiction that there is an adversary $\mathcal{A}$ that wins the experiment with a non-negligible probability. We will describe a simulator $\mathcal{S}$ that uses $\mathcal{A}$ internally, and is indistinguishable from the real execution. We use $\mathcal{S}$ to break the discrete logarithm assumption.

We conceptually expand weights into unit-weight virtual parties: let $I^\star = \{(j,u) : j \in I, u \in [1, w_j]\}$ be the virtual index set of size W, with canonical evaluation points $\{x_{(j,u)}\}_{(j,u)\in I^\star}$. Throughout, for a set of physical parties $X \subseteq I$ we write $X^\star = \{(j,u) : j \in X,\ u \in [1, w_j]\}$ for its expansion into virtual indices, and we reserve primes (I', A', $(A')^\star$, etc.) for the corresponding sets of the new committee after a resharing. Each virtual index (j,u) is owned by party P_j and uses the encryption key pk_j; we write $\mathsf{owner}(j,u) = j$. Let $A \subseteq I$ be the set of corrupted parties of total weight at most f, and define the corrupted virtual indices $A^\star = \{(j,u) : j \in A, u \in [1, w_j]\}$ and $H^\star = I^\star \setminus A^\star$. Let $H = I \setminus A$ be the set of honest physical parties. Let $a = |A^\star| \le f$. Introduce $f - a$ private dummy interpolation indices Z at distinct nonzero points outside the recipient set, and write $B = A^\star \cup Z$. For every honest dealing, independently sample evaluations at these additional points and use them together with the corrupted evaluations when interpolating. The same B is used for DKG and nonce dealings within an epoch; the new epoch uses $B' = (A')^\star \cup Z'$ of size f'. Dummy evaluations are never encrypted, transmitted, or treated as corruptions. A constant and the f evaluations at B uniquely specify a degree-f polynomial, with the correct uniform distribution. We assume $q > W + f + 1$ in each epoch so enough distinct points exist.

Setup

$\mathcal{S}$ receives from the discrete logarithm challenger vk^*. For each honest physical party $P_i \in H$, $\mathcal{S}$ generates an encryption key pair $(\mathsf{sk}_i, \mathsf{pk}_i)$ and publishes pk_i. The adversary $\mathcal{A}$ provides the public keys $\{\mathsf{pk}_j\}_{j\in A}$ for corrupted parties. $\mathcal{S}$ then emulates the honest parties as follows.

Throughout, a physical party certifies with its weight only after all of its virtual shares pass the applicable checks; it signs at most one dealing digest per dealer/session. Messages awaiting certificate or selection prerequisites are buffered. These conventions apply to the virtual-party steps below, including the skipped honest-dealer checks.

DKG Simulation

As in the DKG (Section 3), dealers are *physical* parties: each dealer contributes a single degree-f polynomial evaluated at all virtual points, while recipients are virtual parties. Accordingly, $\mathcal{S}$ runs Phase 1 once for each honest physical dealer $P_i \in H$, and Phases 2 and 3 once for each honest virtual party $(i,v) \in H^\star$.

1. Phase 1 (honest physical dealer $P_i \in H$):
   (a) Sample $\delta_i \leftarrow \mathbb{Z}_q$ and $s_{i,(j,u)} \leftarrow \mathbb{Z}_q$ for each $(j,u) \in A^\star$. Also, sample $s_{i,z} \leftarrow \mathbb{Z}_q$ for $z \in Z$. Set $s_{i,(j,u)} = 0$ for each $(j,u) \in H^\star$.
   (b) Set $C_{i,0} \leftarrow \mathsf{vk}^* \cdot g^{\delta_i}$ and compute $C_{i,k}$ corresponding to coefficients of $x^k, k \in [1,f]$ in the implicitly defined dealing polynomial p_i, by interpolation in the exponent from the coordinates $(0, C_{i,0})$ and $(x_{(j,u)}, g^{s_{i,(j,u)}})$ for all $(j,u) \in A^\star$, together with $(x_z, g^{s_{i,z}})$ for $z \in Z$.
   (c) Compute $E_i = \mathsf{Enc}(\mathsf{sid}_i, PK^\star, S_i)$ where $PK^\star = \{\mathsf{pk}_{\mathsf{owner}(j,u)} : (j,u) \in I^\star\}$ and $S_i = (s_{i,(j,u)})_{(j,u)\in I^\star}$.
   (d) Send to all parties the tuple $(C_{i,0}, C_{i,1}, \ldots, C_{i,f}, E_i)$.
   (e) Wait for signatures on the hash of the message from virtual parties of total weight $2f+1$ and post the aggregate certificate σ_i to the TOB.
2. Phase 2 (honest virtual party $(i,v) \in H^\star$, receiver):
   (a) For a dealer message received from an honest P_j: sign the hash of the message and send the signature to P_j (the simulator skips verification for honest dealers, whose components for honest recipients encrypt zero shares).
   (b) For a dealer message received from an adversarial P_j:
      i. Perform the sanity checks and decrypt the (i,v)-th component of E_j using sk_i.
      ii. If the result was $\perp$ (meaning that decryption failed), ignore the message.
      iii. Else,
         - Let $s_{j,(i,v)}$ be the decrypted message.
         - Verify that $g^{s_{j,(i,v)}} = \prod_{k=0}^{f} C_{j,k}^{x_{(i,v)}^k}$.
         - If valid, store $s_{j,(i,v)}$, sign the hash of the message, and send the signature to P_j. Otherwise, ignore the message.
3. Phase 3 (honest virtual party $(i,v) \in H^\star$, dealer selection and completeness):
   (a) Let I_{sel} be the set of the first physical dealers, in TOB order, that posted valid aggregate certificates σ_j, whose total weight reaches $f+1$.
   (b) For each $j \in I_{\mathrm{sel}} \cap A$ such that (i,v) is not a signer of σ_j: decrypt and verify as in Phase 2; if any check fails, broadcast the complaint $F_{(i,v),j} = \mathsf{Reveal}(\mathsf{sid}_j, \mathsf{sk}_i, (i,v), E_j)$, wait for already-validated shares of dealer j from signers of σ_j of total weight $f+1$, and interpolate them to recover $s_{j,(i,v)}$. (For $j \in I_{\mathrm{sel}} \cap H$ the simulator skips these checks, as in Phase 2.)
   (c) Upon a complaint $F_{(j,u),j'}$ on a dealer $j' \in I_{\mathrm{sel}}$ from a virtual party (j,u): parse $F_{(j,u),j'}$ as $(s_{j',(j,u)}, \pi)$; if $\mathsf{Vf}(\mathsf{sid}_{j'}, \mathsf{pk}_j, s_{j',(j,u)}, \pi, (j,u), E_{j'})$ passes, either $s_{j',(j,u)} = \perp$ or $g^{s_{j',(j,u)}} \neq \prod_{k=0}^{f} C_{j',k}^{x_{(j,u)}^k}$, and (i,v) is a signer of $\sigma_{j'}$, send (i,v)'s validated share $s_{j',(i,v)}$ to (j,u).
   (d) Output $\mathsf{vk} = \prod_{j\in I_{\mathrm{sel}}} C_{j,0}$ as the joint verification key, and the aggregated VSS verification keys $\{\prod_{j\in I_{\mathrm{sel}}} C_{j,k}\}_{k=0}^{f}$. Record $s_{(i,v)} = \top$ to indicate that virtual party (i,v) holds a valid share for vk (the simulator cannot compute the scalar shares of honest virtual parties).

Each dealer $j \in I_{\mathrm{sel}} \cap A$ posted a certificate with signatures of total weight $2f+1$, of which weight at least $2f+1-f = f+1$ comes from honest virtual parties, and an honest virtual party signs only after validating its decrypted share of dealer j. Thus the simulator $\mathcal{S}$ collected valid shares of dealer j from virtual parties of total weight at least $f+1$, and it can use those to recover the full polynomial $\mathsf{p}_j(\cdot)$ and $c_{j,0} = \mathsf{p}_j(0)$. We also emphasize here that the set I_{sel} is the same for all honest virtual parties, since aggregate certificates are ordered by the TOB.

Observe that the final VSS keys commit to a polynomial $\mathsf{p}'(\cdot)$ with coefficients

$$c_k = \sum_{j\in I_{\mathrm{sel}}^H} c_{j,k} + \sum_{j\in I_{\mathrm{sel}}^A} c_{j,k} \quad \text{for } k \in [0,f]\,,$$

where $I^H_{\text{sel}} = I_{\text{sel}} \cap H$ (honest physical dealers in I_{sel}) and $I^A_{\text{sel}} = I_{\text{sel}} \cap A$ (adversarial physical dealers in I_{sel}).

For honest dealers $j \in I^H_{\text{sel}}$, we have $c_{j,0} = x^* + \delta_j$, where x^* is the secret key that corresponds to vk^* (from the construction $C_{j,0} = \mathsf{vk}^* \cdot g^{\delta_j}$). The simulator knows the shares $s_{j,(i,v)}$ at the corrupted points but not the scalar coefficients $c_{j,k}$, which depend on x^*; it knows the coefficients only in the exponent, through the posted commitments $C_{j,k}$. Thus, we have:

$$\begin{aligned}
\mathsf{vk} &= g^{\sum_{j \in I_{\text{sel}}} c_{j,0}} \\
&= g^{\sum_{j \in I^H_{\text{sel}}} (x^* + \delta_j) + \sum_{j \in I^A_{\text{sel}}} c_{j,0}} \\
&= (\mathsf{vk}^*)^{|I^H_{\text{sel}}|} g^{\sum_{j \in I^H_{\text{sel}}} \delta_j + \sum_{j \in I^A_{\text{sel}}} c_{j,0}} \\
&= (\mathsf{vk}^*)^{|I^H_{\text{sel}}|} g^{\sum_{j \in I^H_{\text{sel}}} \delta_j + \sum_{j \in I^A_{\text{sel}}} \sum_{(i,v) \in T^\star_j} \lambda_{0,T^\star_j,(i,v)} \cdot s_{j,(i,v)}}
\end{aligned}$$

where, for each adversarial dealer $j \in I^A_{\text{sel}}$, $T^\star_j$ denotes a set of virtual indices of total weight $f+1$ whose valid shares of dealer j the simulator collected, and $\lambda_{0,T^\star_j,(i,v)}$ is the Lagrange coefficient at point 0 with respect to the evaluation points $\{x_{(i,v)}\}_{(i,v) \in T^\star_j}$.

For virtual party (i,v)'s verification key $\mathsf{vk}_{(i,v)}$, we have $\mathsf{vk}_{(i,v)} = g^{s_{(i,v)}}$ where $s_{(i,v)} = \sum_{j \in I_{\text{sel}}} s_{j,(i,v)}$ is virtual party (i,v)'s share. Using the commitment relation $g^{s_{j,(i,v)}} = \prod_{k=0}^{f} C_{j,k}^{x_{(i,v)}^k}$, we directly obtain

$$\mathsf{vk}_{(i,v)} = \prod_{j \in I_{\text{sel}}} g^{s_{j,(i,v)}} .$$

This expression uses only posted commitments and does not require the simulator to manipulate scalar shares explicitly. Together with the expression for vk above,

$$\mathsf{vk} = (\mathsf{vk}^*)^{|I^H_{\text{sel}}|} \cdot g^{\sum_{j \in I^H_{\text{sel}}} \delta_j + \sum_{j \in I^A_{\text{sel}}} \sum_{(i,v) \in T^\star_j} \lambda_{0,T^\star_j,(i,v)} \, s_{j,(i,v)}} ,$$

this yields a representation entirely in terms of vk^*, the offsets δ_j, and shares $s_{j,(i,v)}$ that the simulator can compute.

We denote $\tau := |I^H_{\text{sel}}|$ as the number of honest physical dealers in I_{sel} (note that $\tau \geq 1$, since the dealers in I_{sel} have total weight at least $f+1$ while the corrupted weight is at most f), and

$$w := \sum_{j \in I^H_{\text{sel}}} \delta_j + \sum_{j \in I^A_{\text{sel}}} \sum_{(i,v) \in T^\star_j} \lambda_{0,T^\star_j,(i,v)} \, s_{j,(i,v)}$$

as the known offset, so that $\mathsf{vk} = (\mathsf{vk}^*)^\tau \cdot g^w$. Crucially, this invariant is preserved across all resharing epochs: since resharing reconstructs the same master secret via Lagrange interpolation (not by summing shares), the relationship between vk and vk^* remains unchanged.

The simulator also knows the aggregate key evaluation s_z at every $z \in Z$: honest dealers' evaluations there were sampled, and selected corrupted DKG polynomials were extracted. These are private interpolation values, not additional exposed shares. Since $1 \leq \tau \leq n < q$, τ is invertible.

Resharing Simulation (epoch $e \to e+1$)

On a resharing query $\mathcal{O}_{\mathsf{reshare}}(I', \{w'_j\}, A')$, the simulator transitions from the current committee $(I, \{w_j\}, A)$ at epoch e to the new committee at epoch $e+1$. Let $(I')^\star = \{(j,u) : j \in I', u \in [1, w'_j]\}$ be the new virtual index set, $(A')^\star = \{(j,u) : j \in A', u \in [1, w'_j]\}$ the corrupted virtual parties in the new committee, and $(H')^\star = (I')^\star \setminus (A')^\star$ the honest new virtual parties. Let $H' = I' \setminus A'$ be the set of honest physical parties in the new committee. Let $I^\star_{\text{dealers}} = I^\star$ be the set of old virtual parties who act as dealers (the entire old committee; a dealer need not remain in the new committee, so the deployment restriction of Section 3.2 is a special case).

For each honest physical party $P_i \in H'$, $\mathcal{S}$ generates a fresh encryption key pair $(\mathsf{sk}'_i, \mathsf{pk}'_i)$ and publishes pk'_i. The adversary provides keys $\{\mathsf{pk}'_j\}_{j \in A'}$ for corrupted parties.

Old Committee (Dealers): Unlike the DKG, resharing uses *virtual* dealers, since each virtual index reshares its own share (Section 3.2). Each virtual party $(i,v) \in I^\star_{\text{dealers}}$ shares its existing DKG share $s_{(i,v)}$. For each *honest* old virtual party $(i,v) \in I^\star_{\text{dealers}} \cap H^\star$:

1. The constant term of the resharing polynomial is the existing share $s_{(i,v)}$, so set the commitment $C'_{(i,v),0} \leftarrow \mathsf{vk}_{(i,v)} = g^{s_{(i,v)}}$ (no additional randomness, this is the dealer's existing share).

2. Sample $s'_{(i,v),(j,u)} \leftarrow \mathbb{Z}_q$ for each $(j,u) \in (A')^\star$. Also sample $s'_{(i,v),z} \leftarrow \mathbb{Z}_q$ for $z \in Z'$. Set $s'_{(i,v),(j,u)} = 0$ for each $(j,u) \in (H')^\star$.

3. Compute $C'_{(i,v),k}$ for $k \in [1, f']$ by interpolating the polynomial over $(0, C'_{(i,v),0})$ and $(x'_{(j,u)}, g^{s'_{(i,v),(j,u)}})$ for all $(j,u) \in (A')^\star$, together with $(x'_z, g^{s'_{(i,v),z}})$ for $z \in Z'$, where $\{x'_{(j,u)}\}$ are the canonical evaluation points for the new committee.

4. Compute $E'_{(i,v)} = \mathsf{Enc}(\mathsf{sid}_{(i,v)}, (PK')^\star, S'_{(i,v)})$ where $(PK')^\star = \{\mathsf{pk}'_{\mathsf{owner}(j,u)} : (j,u) \in (I')^\star\}$ and $S'_{(i,v)} = (s'_{(i,v),(j,u)})_{(j,u)\in(I')^\star}$.

5. Send to all parties in I' the tuple $(C'_{(i,v),0}, \ldots, C'_{(i,v),f'}, E'_{(i,v)})$, wait for signatures on its hash from new virtual parties of total weight $2f'+1$, and post the aggregate certificate to the TOB.

New Committee (Recipients): The honest new virtual parties $(j,u) \in (H')^\star$ proceed through the verification, certification, and completeness phases analogously to the DKG simulation, using the new evaluation points $\{x'_{(j,u)}\}$; in particular, honest recipients skip verification of honest dealers' messages and verify only adversarial dealers' messages. Each recipient additionally verifies that $C'_{(i,v),0}$ matches the expected partial verification key $\mathsf{vk}_{(i,v)}$ from the previous configuration. As in the DKG simulation, for every adversarial dealer whose message was accepted, $\mathcal{S}$ recovers the full resharing polynomial from the shares decrypted by honest new virtual parties of total weight at least $f'+1$; combined with the shares it sampled on behalf of honest dealers, $\mathcal{S}$ thus learns the new shares $s'_{(j,u)}$ of all corrupted virtual parties $(j,u) \in (A')^\star$.

Finalization: Let $I^\star_{\mathrm{sel}} \subseteq I^\star_{\mathrm{dealers}}$ be the set of the first old virtual parties, in TOB order, with valid aggregate certificates, of total weight $f+1$. The key continuity check verifies using Lagrange interpolation:

$$\prod_{(i,v)\in I^\star_{\mathrm{sel}}} (C'_{(i,v),0})^{\lambda_{0,(i,v)}} = \mathsf{vk}$$

where $\lambda_{0,(i,v)} = \lambda_{0,I^\star_{\mathrm{sel}},(i,v)}$ are the Lagrange coefficients for interpolating at point 0 using evaluation points $\{x_{(i,v)}\}_{(i,v)\in I^\star_{\mathrm{sel}}}$ from the *old* committee.

Since $C'_{(i,v),0} = \mathsf{vk}_{(i,v)}$ for all dealers (honest dealers by construction, adversarial dealers by the additional receiver check), we have (writing $s := \log_g \mathsf{vk}$ for the master secret):

$$\prod_{(i,v)\in I^\star_{\mathrm{sel}}} (\mathsf{vk}_{(i,v)})^{\lambda_{0,(i,v)}} = g^{\sum_{(i,v)\in I^\star_{\mathrm{sel}}} \lambda_{0,(i,v)}\cdot s_{(i,v)}} = g^s = \mathsf{vk}$$

This is exactly the Lagrange reconstruction of the master secret s from the shares $\{s_{(i,v)}\}$, confirming key continuity.

Invariant Preservation: The joint key vk is unchanged by resharing, so the invariant $\mathsf{vk} = (\mathsf{vk}^*)^\tau \cdot g^w$ established during the DKG is automatically preserved. The simulator does not need to update τ or w across resharing operations.

It computes each new private key evaluation s'_z, $z \in Z'$, by applying the same Lagrange coefficients to the sampled honest-dealer and extracted corrupted-dealer evaluations. Thus the invariant also includes known evaluations at all f' points of B'. Under the transition interface, old-role secret state is never exposed by a new-role corruption. This gives the induction needed to simulate later epochs, in addition to public-key continuity.

Finally, update the committee state: $I \leftarrow I'$, $\{w_j\} \leftarrow \{w'_j\}$, $W \leftarrow W'$, $f \leftarrow f'$, $A \leftarrow A'$, $H \leftarrow H'$, $H^\star \leftarrow (H')^\star$, $A^\star \leftarrow (A')^\star$, $\{x_{(j,u)}\} \leftarrow \{x'_{(j,u)}\}$, and $\mathsf{vk}_{(j,u)} \leftarrow \mathsf{vk}'_{(j,u)}$, the new partial verification keys computed from the posted resharing commitments by Lagrange interpolation in the exponent; set $e \leftarrow e+1$. Also, replace B, Z and their evaluation points by B', Z' and the new points, and store the known new key evaluations s'_z as the current s_z.

Nonce Generation Simulation (per sid) The Batch AVSS protocol (Section 4) specifies that dealers are physical parties with weight, while recipients are virtual parties. Each dealer D with weight w_D distributes $w_D L$ nonces to all virtual parties in $I^\star$, where L is the batch size parameter of Section 4.

Dealer Simulation for honest dealer D with weight w_D

1. Sample $u_l, v_l \leftarrow \mathbb{Z}_q$ for each $l \in [1, w_D L]$, and sample $c'_0 \leftarrow \mathbb{Z}_q$.
2. For each $l \in [1, w_D L]$, compute $C_l \leftarrow g^{u_l} \mathsf{vk}^{v_l}$.
3. Sample shares for adversarial and honest recipients:

$$r_{l,(i,v)}, r'_{(i,v)} \begin{cases} \leftarrow \mathbb{Z}_q & \text{for } (i,v) \in A^\star \\ = 0 & \text{for } (i,v) \in H^\star \end{cases}$$

   Also sample $r_{l,z}, r'_z \leftarrow \mathbb{Z}_q$ independently for $z \in Z$.
4. Create the multi-recipient ciphertext $E \leftarrow \mathsf{Enc}(\mathsf{sid}, PK^\star, \langle R_{(i,v)} \rangle_{(i,v)\in I^\star})$ where $PK^\star = \{\mathsf{pk}_i : (i,v) \in I^\star\}$ and $R_{(i,v)} = (r_{1,(i,v)}, \ldots, r_{w_D L,(i,v)}, r'_{(i,v)})$ for all $(i,v) \in I^\star$.
5. Sample $\gamma_l \leftarrow \mathbb{Z}_q$ uniformly for each $l \in [1, w_D L]$.
6. Compute $C' \leftarrow g^{c'_0} \cdot \mathsf{vk}^{-\sum_{l=1}^{w_D L} \gamma_l v_l}$.
7. Program $\mathsf{H}(\mathsf{sid}, l, \langle C_l \rangle_{l\in[1,w_D L]}, C', E)$ to return γ_l for each l. Abort if any such input is already defined; program the whole vector before publishing any part, digest, or fragment of the tuple. The fresh unpublished C' bounds this event as shown in Game 2.
8. Interpolate the degree-f polynomial $\mathsf{p}''(X)$ over the following $|A^\star| + 1$ points, together with $\left(x_z, r'_z + \sum_l \gamma_l r_{l,z}\right)$ for $z \in Z$, giving $f + 1$ points in total:

$$\left(0, c'_0 + \sum_{l=1}^{w_D L} \gamma_l \, u_l\right), \left(x_{(i,v)}, r'_{(i,v)} + \sum_{l=1}^{w_D L} \gamma_l \, r_{l,(i,v)}\right)_{(i,v)\in A^\star}.$$

9. Reliable broadcast $(\langle C_l \rangle_{l\in[1,w_D L]}, C', E, \mathsf{p}''(\cdot))$.
10. Wait for signatures from virtual parties whose total weight reaches $2f+1$ on the hash of the above message and form a certificate.
11. Post the certificate to the TOB.

For each honest virtual party $(i,v) \in H^\star$:

1. For a message from an honest dealer D, sign the broadcast for certification directly: as in the DKG simulation, the simulator skips verification for honest dealers, whose components for honest recipients encrypt zero shares. For a message from an adversarial dealer D do:
   (a) Perform sanity checks (sizes, group membership, etc.). Decrypt the (i,v)-th component of E to obtain $(\langle r_{l,(i,v)} \rangle_{l\in[1,w_D L]}, r'_{(i,v)})$; if decryption returns $\perp$, ignore the message.
   (b) Recompute $\{\gamma_l\}$ and verify $\mathsf{p}''(x_{(i,v)}) = r'_{(i,v)} + \sum_{l=1}^{w_D L} \gamma_l r_{l,(i,v)}$ and $g^{\mathsf{p}''(0)} = C' \cdot \prod_{l=1}^{w_D L} C_l^{\gamma_l}$. If valid, store the decrypted shares and sign the broadcast for certification; otherwise ignore.
2. Upon receiving a valid aggregate certificate for dealer D:
   (a) If (i,v)'s signature is present or the dealer is honest, do nothing. Otherwise, decrypt and verify as above; if any check fails, compute a fraud complaint $F_{(i,v)} := \mathsf{Reveal}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$ and broadcast $F_{(i,v)}$.

(b) Wait for valid reveal responses that also satisfy the folded evaluation check for the same certified p'' from virtual parties whose total weight reaches $f+1$ and interpolate to recover the missing shares for (i, v) (retrying $F_{(i,v)}$ periodically if needed). Store the recovered shares if interpolation succeeds.

3. Upon a fraud complaint $F_{(j,u)}$ from another virtual party:

   (a) If no aggregate certificate for D exists or the complaint is invalid, ignore. Otherwise, compute $F_{(i,v)} := \mathsf{Reveal}(\mathsf{sid}, \mathsf{sk}_i, (i, v), E)$ and send it to (j, u).

By reliable broadcast, all honest virtual parties agree on a unique accepted tuple per sid. Subject to the consistency event below, reveal soundness and eventual delivery let every honest virtual party reconstruct and store its nonce-shares. Terminal cancellation occurs only at an agreed epoch retirement.

For each adversarial dealer whose certificate is accepted, the $2f + 1$ signature weight includes honest virtual parties of total weight at least $f + 1$ that verified their decrypted shares. Except for an event of total probability $\varepsilon_{\text{fold}}$ defined below, these shares and every folded-valid recovery response are consistent with degree-f polynomials p_l satisfying $g^{\mathsf{p}_l(0)} = C_l$. This consistency property is used for liveness. For unforgeability, $\mathcal{S}$ instead keeps each honest recipient's actual decrypted or recovered values and follows the real recovery procedure, even when different recipients' values are inconsistent.

For completeness, in epoch e let $Q_{\text{fold},e}$ count all distinct tuples $(\mathsf{sid}, \langle C_l \rangle, C', E)$ on which a folding coordinate is queried, including unsuccessful candidates. Let n_e be the number of physical parties. Then

$$\varepsilon_{\text{fold}} \le \min\left\{1, \sum_e \frac{Q_{\text{fold},e}}{q} \min\left\{\binom{W_e}{f_e+1}, 2^{n_e}(W_e+1)\right\}\right\}.$$

Fix such a tuple immediately before its first folding query. Its uniquely bound plaintexts at all positions are fixed. For each set T of $f+1$ positions, interpolate $p_{T,l}$ and $p_{T,0}$ from its nonce and mask plaintexts, and put $d_{T,l} = p_{T,l}(0) - \log_g C_l$ and $d_{T,0} = p_{T,0}(0) - \log_g C'$. These logarithms are for analysis only. Acceptance at T forces the published polynomial to be $p_{T,0} + \sum_l \gamma_l p_{T,l}$; the constant check then forces $d_{T,0} + \sum_l \gamma_l d_{T,l} = 0$. A nonzero discrepancy satisfies this equation with probability at most $1/q$ for the fresh vector. Union over all T and all candidate tuples yields the bound, even when T and the published polynomial are chosen after the vector.

The second bound uses the physical-party acceptance rule of Section 4. For each set of physical signers of weight at least $f + 1$, choose a fixed $f + 1$-point interpolation subset. Charge $1/q$ for its constant discrepancy and at most W/q for any other original, uniquely bound recipient plaintext that differs from these polynomials yet passes the folded check. The actual honest certificate signers are one of at most 2^n such sets; this also covers folded-valid recovery responses.

Outside this event, all folded-valid positions agree with one set of polynomials: replace one position in a base set of $f+1$ accepting positions by any other accepting position. The interpolated polynomials share f evaluations and the committed constant at zero, and hence are identical. A certificate supplies a base set with at least $f + 1$ honest positions. Recovery therefore reconstructs the same polynomials. The unforgeability simulator does not abort on this event: it retains the actual recipient values. Consequently, the bound above is charged only in the liveness analysis.

Presigning Simulation

For presigning, the simulator proceeds as follows. Recall that dealers are physical parties, and each dealer j with weight w_j contributes $w_j L$ nonce commitments.

1. Wait until valid certificates are posted on the TOB by dealers whose total weight reaches $2f+1$ (optionally wait an additional grace period to accumulate more). Use the common TOB cutoff and unique $\mathsf{PresigDealerSet}(\eta, \mathsf{bid}, J)$ record of Section 5.1. The selected dealings determine D and S before β_p is disclosed. Let J be the set of certified dealers (physical parties), let $W_J = \sum_{j \in J} w_j$, and set $M := W_J - f$.

2. Form the public commitment matrix $C \in \mathbb{G}^{W_J \times L}$ by stacking the dealer commitments: for each dealer $j \in J$ with weight w_j, arrange the $w_j L$ commitments $(C_{j,l})_{l \in [1, w_j L]}$ from dealer j's Batch AVSS into w_j rows of L entries, so that $C_{(j,u),l} := C_{j,(u-1)L+l}$. The rows are indexed by the expanded dealer set $J^\star = \{(j, u) : j \in J, u \in [1, w_j]\}$.

3. Derive the SI matrix $S \in \mathbb{F}^{M \times W_J}$, where $\mathbb{F} = \mathbb{Z}_q$, from the ordered set $J^\star$, where S is super-invertible: any M of its W_J columns are linearly independent.

4. Compute and store $D := S \cdot C \in \mathbb{G}^{M \times L}$. For online signing, retain the actual contribution from corrupted dealers in each honest recipient's state; honest-dealer contributions are represented implicitly below.

5. Compute the auxiliary scalar matrices indexed by $(j,u) \in J^\star$ and $l \in [1, L]$:

$$U'_{(j,u),l} := \begin{cases} u_{(j,u),l} & \text{if } j \in J \cap H \\ 0 & \text{otherwise} \end{cases} \text{ and } V'_{(j,u),l} := \begin{cases} v_{(j,u),l} & \text{if } j \in J \cap H \\ 0 & \text{otherwise} \end{cases}$$

   where $H = I \setminus A$ is the set of honest physical parties, $u_{(j,u),l}, v_{(j,u),l}$ are the values sampled in the Dealer Simulation for honest dealer j (re-indexed as $u_{(j,u),l} := u_{j,(u-1)L+l}$ and $v_{(j,u),l} := v_{j,(u-1)L+l}$, as for C). Thus U' and V' represent only honest-dealer contributions.

6. Compute $P' \in \mathbb{F}[X]^{W_J \times L}$ as follows. For honest dealers, $P'_{(j,u),l}$ interpolates $(0, u_{(j,u),l})$ and the sampled evaluations at all f points of B; for corrupted dealers set $P'_{(j,u),l} = 0$.

7. Store the tuple $(U, V, P) \leftarrow (S \cdot U', S \cdot V', S \cdot P')$. Here $g^U \mathsf{vk}^V$ is the honest-dealer contribution to D; no such representation is required for corrupted commitments. Since the corrupted weight in J is at most f, at least $M = W_J - f$ dealer columns are honest; by super-invertibility the corresponding $M \times M$ submatrix is nonsingular, the selected rows give independent linear forms in the hidden v_l. Their joint uniformity, including the information in earlier challenges and scalar partials, is established in Game 5; marginal uniformity alone would not suffice. At the first disclosure of each batch seed, compute all its masked slot commitments $A_j = D_j g^{\delta_{p,j}}$, including unused slots, and abort if any coincides with an earlier or simultaneous masked slot commitment. Game 5 bounds this event.

Message Signing Simulation

Let ℓ_B be the degree-f polynomial with $\ell_B(0) = 1$ and $\ell_B(x_b) = 0$ for $b \in B$. Let K_0 interpolate $K_0(0) = 0$ and the known current key evaluations $K_0(x_b) = s_b$. For analysis, the real key polynomial is $K = K_0 + x\ell_B$, where $x = \log_g \mathsf{vk}$, and the combined honest nonce polynomial for slot (t,l) is $P_{t,l} + xV_{t,l}\ell_B$. Write $a_{t,l,(i,v)}$ for the weighted sum of the corrupted dealers' actual nonce values in honest recipient (i,v)'s state; the simulator knows these values without imposing polynomial consistency.

1. On a query to sign message m using an unused coordinate (sid, t, l) where $t \in [1, M]$ and $l \in [1, L]$: Perform the following beacon/programming steps when the last designated fresh seed is first disclosed, jointly for all requests relying on that disclosure; the request and slot are already fixed.

   (a) Sample the last fresh beacon seed according to its real distribution, retain the earlier seed and its actual PRF values, compute $\delta = \delta_k + \delta_p$ using the actual PRF, and set $R_{t,l} \leftarrow (D)_{t,l} g^\delta$.

   (b) Program the random oracle at $(m, R_{t,l}, \mathsf{vk})$ to $e := -(V)_{t,l}$. Handle prior queries and coincident inputs as in Game 5; all points for this disclosure are processed before the seed is exposed.

   (c) Only then release the actual last beacon seed. Both public seeds and every later PRF evaluation remain exactly consistent with the protocol. Releasing that seed earlier would permit a rushing query before programming.

   (d) For each requested honest virtual party $(i,v) \in H^\star$ that has obtained the required nonce values, return $\sigma_{(i,v)} = P_{t,l}(x_{(i,v)}) + a_{t,l,(i,v)} - V_{t,l} K_0(x_{(i,v)})$. These are exactly its real partials: the terms $xV_{t,l}\ell_B(x_{(i,v)})$ from the honest nonce and $-V_{t,l}x\ell_B(x_{(i,v)})$ from eK cancel.

2. The signature aggregation steps are identical to the ones in the real protocol, using evaluation points $\{x_{(i,v)}\}$ for interpolation.

The simulator processes arbitrary corrupted partials with the real aggregation and verification rules. Inconsistent nonce values can prevent completion; the simulated execution then remains pending or is retired exactly as the real execution. Unforgeability does not require forcing a valid aggregate.

Forgery

Let $\mathcal{E}$ be the event that $\mathcal{A}$ outputs a valid Schnorr signature (R^*, s^*) on some unqueried message m^* under the joint verification key vk, and that $\mathcal{A}$ queried the random oracle at (m^*, R^*, vk). If $\mathsf{vk} = 1$, the simulator immediately extracts $x^* = -w\tau^{-1}$. Otherwise, a valid forgery without that query has probability at most $1/q$, so $\Pr[\mathcal{E}] \geq \varepsilon - 1/q$ where ε is $\mathcal{A}$'s success probability in the simulated execution; by the indistinguishability argument of the next subsection, ε differs from $\mathcal{A}$'s success probability in the real experiment by at most $\mathsf{negl}(\lambda)$.

Conditioned on $\mathcal{E}$, we apply the Forking Lemma: we rewind $\mathcal{A}$ with the same internal randomness and replay the complete prefix and reply to the forgery query (m^*, R^*, vk) with a fresh independent challenge $e_1 \leftarrow \mathbb{Z}_q$; subsequent fresh Schnorr-oracle answers are resampled, other oracle tapes are replayed, and the simulator continues consistently. All oracle points programmed by the simulator before the forking point (the values $\{\gamma_l\}$ from nonce generation and the signing challenges $-V_{t,l}$) are replayed identically across both runs. Writing $p = \Pr[\mathcal{E}]$, the general forking lemma gives success probability at least $p(p/Q_{\mathsf{H}} - 1/q)$ when positive, where $Q_{\mathsf{H}} \geq 1$ bounds all Schnorr-oracle evaluations, including protocol evaluations. With that probability, we obtain two accepting transcripts for the same (m^*, R^*, vk) with distinct challenges $e_0 \neq e_1$:

$$g^{s_0^*} = R^* \cdot \mathsf{vk}^{e_0}\,, \qquad\qquad g^{s_1^*} = R^* \cdot \mathsf{vk}^{e_1}\,.$$

We extract the discrete logarithm of vk as

$$x := (s_0^* - s_1^*) \cdot (e_0 - e_1)^{-1} \bmod q\,; \quad \text{then } \mathsf{vk} = g^x\,.$$

By the DKG simulation above, we have the invariant

$$\mathsf{vk} = (\mathsf{vk}^*)^{\tau} \cdot g^{w}\,,$$

where $\tau = |I^H_{\mathrm{sel}}| \geq 1$ (the number of honest physical dealers selected in the DKG) and w is computed from the DKG simulation. Crucially, this invariant is preserved across all resharing operations: since resharing uses Lagrange interpolation to reconstruct shares of the *same* master secret, the relationship between vk and vk^* remains unchanged regardless of how many resharings occur.

Since q is prime and $\tau < q$, τ is invertible in $\mathbb{Z}_q$, and thus

$$x = \tau\, x^* + w \bmod q \quad \Longrightarrow \quad x^* = (x - w) \cdot \tau^{-1} \bmod q\,.$$

Therefore $\mathcal{S}$ recovers the discrete logarithm of vk^* from the forgery.

Coincident programming inputs instead cause the bounded abort analyzed in Game 5. A forgery on an unqueried message cannot target a signing-programmed point. If the forked query precedes DKG finalization, two successful forks still have the same public key in that query; the extraction uses τ, w from the first successful branch, without requiring the later dealer selections to coincide.

6.2 Proof of Indistinguishability

We prove indistinguishability by a sequence of hybrid games. Throughout, the network is asynchronous with adversarially scheduled (rushing) delivery, subject to the TOB and reliable-broadcast guarantees; corruptions are epoch-wise static (the corrupted set may change only at resharing queries), the random oracle (RO) is modeled as a lazy-sampled function programmable at points chosen by the simulator, and the multi-recipient encryption used in DKG/nonce generation/resharing satisfies ADO-CCA [GS22] (associated-data-only CCA, with the multi-challenge and reveal interfaces specified above). The intermediate games are defined as distributions and may use the real secret key and shares (so honest signing is well defined in each); only the final game coincides with the discrete-logarithm reduction's simulator, which is independent of x^*. The last transition (Game 4 to Game 5) removes the dependence on the secret from online signing.

Remark on Resharing. The resharing protocol follows a similar structure to the DKG: old virtual parties act as dealers distributing their existing shares to new virtual parties. The key difference is that in resharing, each dealer's constant term is their existing share $s_{(i,v)}$ (committed as $\mathsf{vk}_{(i,v)}$), with no additional randomness. The simulator handles resharing queries by setting honest-to-honest shares to zero and sampling random shares for adversarial recipients, while the constant term commitments $C'_{(i,v),0} = \mathsf{vk}_{(i,v)}$ are taken from the current epoch's partial verification keys. Since resharing uses Lagrange interpolation to reconstruct the same master secret, the invariant $\mathsf{vk} = (\mathsf{vk}^*)^\tau \cdot g^w$ is automatically preserved. The indistinguishability argument for resharing is subsumed by the same ADO-CCA arguments used for the DKG. Therefore, the hybrid sequence below implicitly covers any number of resharing queries. We use the isolated old/new state-exposure interface and the known dummy-evaluation invariant above; continuity of vk alone does not justify mobile security.

Game 0 (Real). This is the real execution of the protocol with the real RO. Honest dealers in nonce generation sample their values as specified, broadcast $(\langle C_l\rangle_{l\in[1,w_D L]}, C', E, \mathsf{p}''(\cdot))$, and virtual parties proceed through presigning and online signing.

Game 1 (Lazy RO). We replace the RO by a lazy-sampled oracle that returns fresh uniform values on first query and caches them thereafter. This is identically distributed to the real RO.

Game 2 (Programming hash challenges for nonce generation). For every honest dealer D with weight w_D and session identifier sid, we program the RO values $\{\gamma_l\}_{l\in[1,w_D L]}$ at the input $(\mathsf{sid}, l, \langle C_l\rangle, C', E)$ as follows. We sample $\{u_l\}, \{v_l\}, c'_0 \leftarrow\$ \mathbb{Z}_q$, sample $\{r_{l,(i,v)}\}, \{r'_{(i,v)}\} \leftarrow\$ \mathbb{Z}_q$ for $(i,v) \in A^\star$, set $C_l \leftarrow g^{u_l}\,\mathsf{vk}^{v_l}$, and sample $\{\gamma_l\} \leftarrow\$ \mathbb{Z}_q$ *uniformly*. We then compute $C' \leftarrow g^{c'_0}\,\mathsf{vk}^{-\sum_l \gamma_l v_l}$ and interpolate the degree-f polynomial $\mathsf{p}''(\cdot)$ over the following $|A^\star|+1$ points and the $f-|A^\star|$ sampled dummy points

$$\Big(0,\, c'_0 + \sum_{l=1}^{w_D L} \gamma_l u_l\Big),\ \Big(x_{(i,v)},\, r'_{(i,v)} + \sum_{l=1}^{w_D L} \gamma_l r_{l,(i,v)}\Big)_{(i,v)\in A^\star}$$

and include $(x_z, r'_z + \sum_l \gamma_l r_{l,z})$ for $z \in Z$. Construct the ciphertext, program the folding queries, and then broadcast $(\langle C_l\rangle, C', E, \mathsf{p}''(\cdot))$. Abort if a programmed input was already defined. Before the tuple is exposed, its mask commitment C' is fresh uniform, so the total probability is at most $B_H Q_{\mathrm{nc}}/q$, where B_H counts honest nonce dealings and Q_{nc} counts prior folding queries. This uses the entropy of C', not of the ciphertext. The honest recipients' components of E encrypt the consistent shares, namely the evaluations at the honest points of the degree-f polynomials determined by the constant terms $u_l + x\,v_l$ (for the nonce polynomials) and $c'_0 - x\sum_l \gamma_l v_l = \log_g C'$ (for the masking polynomial) together with the sampled shares at the corrupted points and dummy points, computed using the real secret key $x = \log_g \mathsf{vk}$, as permitted for intermediate games. The recipients' checks hold by construction:

- $\mathsf{p}''(x_{(i,v)}) = r'_{(i,v)} + \sum_l \gamma_l r_{l,(i,v)}$ by interpolation.
- $g^{\mathsf{p}''(0)} = C' \cdot \prod_l C_l^{\gamma_l}$ because:

$$C' \cdot \prod_l C_l^{\gamma_l} = g^{c'_0}\mathsf{vk}^{-\sum_l \gamma_l v_l} \cdot \prod_l g^{\gamma_l u_l}\mathsf{vk}^{\gamma_l v_l} = g^{c'_0 + \sum_l \gamma_l u_l} = g^{\mathsf{p}''(0)}.$$

The transition from Game 1 to Game 2 is *purely information-theoretic*. The key insight is that we compute C' based on the sampled γ values, rather than constraining γ. From the adversary's view:

- Each $C_l = g^{u_l + v_l x}$ reveals $c_l = u_l + v_l x$, leaving v_l uniformly distributed.
- The γ_l values are sampled *unconditionally uniform*.
- $C' = g^{c'_0 - x\sum_l \gamma_l v_l}$ reveals $c' = c'_0 - x\sum_l \gamma_l v_l$. Since c'_0 is uniform and independent, c' is also uniform.

Thus C' is a uniformly random group element (identically distributed to the real protocol), and γ is unconditionally uniform. The distributions match up to the stated programming-abort probability.

This includes the full polynomial/ciphertext transcript: for fixed x, the change of variables from (u_l, v_l, c'_0) to $(c_l = u_l + x v_l, v_l, c' = c'_0 - x\sum_l \gamma_l v_l)$ is bijective, with all evaluations at B independently

sampled. Thus the real nonce and mask polynomials, including their encrypted evaluations, have the required distribution independently of the auxiliary v_l. In an encryption hybrid, first sample γ, construct both real and dummy plaintext vectors using the known secret of that intermediate game, obtain its challenge ciphertext E, and only then program the corresponding folding inputs before publication.

Game 3 (Switching honest recipients' encrypted shares). For each honest dealer in the DKG, in resharing, and in nonce generation, we modify the corresponding multi-recipient ciphertext (E_i, $E'_{(i,v)}$, and E, respectively) so that the honest virtual parties' components encrypt fixed dummy values: zero shares in the DKG and resharing, and $r_{l,(i,v)} = 0$ and $r'_{(i,v)} = 0$ for all $(i, v) \in H^\star$ in nonce generation. Only the communicated ciphertexts change; internal honest signing state still uses the original consistent plaintexts in this intermediate game; in addition, honest recipients skip the validity checks for honest dealers' messages (in Game 2 these checks always pass, so behavior is unchanged) and hence never decrypt or reveal an honest dealer's honest components, and their only observable action for such messages (signing for certification) is unaffected. Skipping these decryptions is also what makes the reduction to ADO-CCA well defined, as the reduction cannot decrypt the challenge components. By ADO-CCA security of the underlying encryption, replacing the encrypted plaintexts for honest recipients is indistinguishable even under chosen-ciphertext and reveal queries. The public constant check and corrupted recipients' checks still pass. Honest recipients' dummy plaintexts need not satisfy the folded evaluation equation; those checks are intentionally skipped. Sound, recipient-bound reveal proofs prevent fabricated complaints from exposing honest-dealer challenge components. All permitted other decryptions/reveals use the stated ADO-CCA interface, and the multi-challenge loss is charged below. Therefore, Game 3 is indistinguishable from Game 2.

Game 4 (Presigning via SI linearization). Let J denote the set of certified dealers (physical parties) with total weight W_J, let $J^\star = \{(j, u) : j \in J, u \in [1, w_j]\}$ be the expanded dealer set, and let $S \in \mathbb{F}^{M\times W_J}$ be the SI matrix derived from $J^\star$ where $M = W_J - f$ and S is super-invertible (any M columns are linearly independent). We represent the honest dealers' nonce contributions by the auxiliary matrices, retaining the actual corrupted-dealer recipient values. Concretely, we compute

$$D := S \cdot C \in \mathbb{G}^{M\times L}, \quad U := S \cdot U', \quad V := S \cdot V', \quad P := S \cdot P',$$

as in the simulator, where C collects the dealers' $\{C_{(j,u),l}\}$, U', V' are the per-dealer auxiliary arrays with $V'_{(j,u),l} = v_{(j,u),l}$ for honest dealers $j \in J \cap H$ and 0 otherwise, and U', P' are the honest-only arrays above; the actual corrupted-dealer values remain in the recipient states. The actual combined nonce value at recipient (i, v) is $P_{t,l}(x_{(i,v)}) + xV_{t,l}\ell_B(x_{(i,v)}) + a_{t,l,(i,v)}$. No folded-consistency assumption is used in this change of representation. Since at least $M = W_J - f$ dealer columns are honest, super-invertibility makes the corresponding $M \times M$ submatrix nonsingular, the honest restriction of S has full row rank. Game 5 establishes the conditional joint uniformity needed for signing. Thus the public presigning transcript in this game is distributed as in the real protocol, and Game 4 is indistinguishable from Game 3.

Game 5 (Online signing by RO programming only). For each honest signing query on message m at unused coordinate (sid, t, l) where $t \in [1, M]$ and $l \in [1, L]$, we run the two-beacon procedure above: sample the actual last seed, compute $\delta = \delta_k + \delta_p$ and $R_{t,l} := D_{t,l}g^\delta$, program $e := -V_{t,l}$, and only then release the actual last seed. We process all reservations relying on that disclosure together. Prior-query conflicts cause an abort; simultaneous collisions are handled below. We keep the actual PRF and its public seeds throughout the transcript. We return the pointwise partials above and use the real aggregation algorithm. Their equality with the real honest responses follows from

$$\begin{aligned} P_{t,l}(x_{(i,v)}) + xV_{t,l}\ell_B(x_{(i,v)}) + a_{t,l,(i,v)} - V_{t,l}K(x_{(i,v)}) \\ = P_{t,l}(x_{(i,v)}) + a_{t,l,(i,v)} - V_{t,l}K_0(x_{(i,v)}). \end{aligned}$$

To justify the joint distribution, fix the actual master secret x for analysis, and write the honest initial nonce constants as $\mathbf{r} = \mathbf{u} + x\mathbf{v}$. The auxiliary $\mathbf{v}$ remains uniform even given the real honest polynomials and masks, by the bijection in Game 2. A set T of consumed rows restricted to honest columns has independent rows: it is a subset of one full-rank SI restriction per block column, and different columns/batches have disjoint nonce variables. The block matrix $\left(\begin{smallmatrix} I & xI \\ 0 & T \end{smallmatrix}\right)$ therefore has full row rank. Given the real honest polynomials, actual corrupted-dealer recipient values, and disclosed challenges,

every honest partial and aggregation result is determined; these disclosures impose no further hidden-$\mathbf{v}$ constraint. This is the joint adaptive-uniformity argument of [Sho25, Sho26]. Pending or aborted requests still consume their rows. This proves the required conditional joint distribution, not just marginal uniformity of $V_{t,l}$.

For the programming failure bound, consider r requests relying on one new seed. Immediately before disclosure, write their effective commitments as $R_i = D_i g^{c_i + \mathsf{PRF}_\beta(z_i)}$, with D_i, c_i, z_i, m_i already fixed. A hidden-key PRF distinguisher simulates the preceding execution, evaluates these inputs, checks only whether a prior Schnorr query is hit or two targets with distinct z_i coincide, and stops before key disclosure. With a random function, each target is uniform and distinct inputs give independent offsets, so this probability is at most $(rQ_{\mathsf{H}} + \binom{r}{2})/q$. Summing these local PRF advantages gives $\varepsilon_{\mathrm{PRF}}$. This argument does not replace the outputs after their key is public.

For repeated message inputs under a shared message beacon, define $A_j = D_j g^{\delta_{p,j}}$ for every generated slot. At the first disclosure of each batch seed, its commitments and distinct inputs $(\eta, \mathsf{bid}, J, t, l, D_{t,l})$ are fixed. A hidden-key PRF test, ending before disclosure, compares all new A_j with earlier ones and with each other. Summed over all N_{pre} generated slots, including unused slots, its random-function collision bound is $\binom{N_{\mathrm{pre}}}{2}/q$; include these local PRF advantages in $\varepsilon_{\mathrm{PRF}}$. Equal message inputs under the same last seed have the same message offset, so a coincident Schnorr input requires such an A_j collision. Abort on either collision type. The rank argument is applied along prefixes before the first bad event, without conditioning on future choices.

At this point the transcript is distributed exactly as in the simulator used for the reduction: in particular, the DKG and resharing commitments produced by the simulator are identically distributed to those of Game 5, since $C_{i,0} = \mathsf{vk}^* \cdot g^{\delta_i}$ is uniform for uniform δ_i, and the remaining commitments are interpolated from uniformly random shares at the corrupted points, matching the distribution induced by a uniformly random polynomial with the given constant term. Let $\varepsilon_{\mathrm{aux}}$ collect encryption, authentication, digest/reveal binding, and ideal-beacon realization errors. Let Q_s count requests whose final fresh seed is disclosed, and include rejected candidates and protocol evaluations in all query bounds. Summing the first-bad-event bounds above gives the simulation error. Therefore,

$$\Big|\Pr[\mathsf{Game}\ 0 = 1] - \Pr[\mathsf{Game}\ 5 = 1]\Big| \le \varepsilon_{\mathrm{aux}} + \varepsilon_{\mathrm{PRF}} + \frac{B_H Q_{\mathrm{nc}} + Q_s Q_{\mathsf{H}} + \binom{Q_s}{2} + \binom{N_{\mathrm{pre}}}{2}}{q}.$$

and the simulated execution is indistinguishable from the real protocol. The bound is negligible for polynomially many dealings, slots and queries under the stated assumptions. It contains no folded-consistency error; that error remains in the liveness bound below.

6.3 Extension to Key Derivation

The security proof extends naturally to the key derivation setting described in Section 5. Recall that from the master MPC public key vk, we derive child keys $\mathsf{vk}_{\mathsf{tag}} = \mathsf{vk} \cdot g^{\mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})}$ for derivation tags $\mathsf{tag} \in \{0,1\}^*$. The adversary may now request signatures under any derived key $\mathsf{vk}_{\mathsf{tag}}$ and attempt to forge a signature under some derived key $\mathsf{vk}_{\mathsf{tag}^*}$ for which the message was not queried.

Adapted Security Experiment. The experiment is modified so that:

1. The signing oracle $\mathcal{O}_{\mathsf{Sign}}(\mathsf{tag}, m)$ takes a derivation tag tag and message m, and returns a Schnorr signature on m under the derived key $\mathsf{vk}_{\mathsf{tag}}$.

2. The adversary outputs $(\mathsf{tag}^*, m^*, \sigma^*)$ and wins if $\mathsf{Vf}(\sigma^*, m^*, \mathsf{vk}_{\mathsf{tag}^*}) = 1$ and the effective-key/message pair $(\mathsf{vk}_{\mathsf{tag}^*}, m^*)$ was never authorized. Distinct tags that derive the same key represent the same signing key.

Simulation Adaptation. The simulator handles derived-key signing queries as follows. On query (tag, m) at coordinate (sid, t, l):

1. Compute the derivation offset $d_{\mathsf{tag}} := \mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})$ and the derived key $\mathsf{vk}_{\mathsf{tag}} := \mathsf{vk} \cdot g^{d_{\mathsf{tag}}}$.

2. Fix the effective key and slot before the last fresh beacon, and compute $\delta = \delta_k + \delta_p$ and $R_{t,l} := D_{t,l} g^{\delta}$ using the same two-beacon procedure.

3. Program the random oracle at $(m, R_{t,l}, \mathsf{vk}_{\mathsf{tag}})$ to $e := -V_{t,l}$. Do so before releasing the last seed, jointly with the other requests relying on that disclosure.

4. Return the pointwise honest partials from the Message Signing simulation, using $e = -V_{t,l}$.

5. Run the real aggregation and add the public correction $\delta + e\, d_{\mathsf{tag}}$ to its candidate response. Output only if the resulting signature verifies.

Whenever aggregation produces a valid base response σ_0 at challenge e, the public correction gives:

$$g^{\sigma_0 + \delta + e d_{\mathsf{tag}}} = D_{t,l} \mathsf{vk}^{e} g^{\delta + e d_{\mathsf{tag}}} = R_{t,l} \mathsf{vk}_{\mathsf{tag}}^{e}.$$

The last equality uses $\mathsf{vk}_{\mathsf{tag}} = \mathsf{vk} \cdot g^{d_{\mathsf{tag}}}$ and the programmed hash value $\mathsf{H}(m, R_{t,l}, \mathsf{vk}_{\mathsf{tag}}) = -V_{t,l}$.

Extraction from Forgery. Suppose the adversary produces a valid forgery (R^*, s^*) on message m^* under derived key $\mathsf{vk}_{\mathsf{tag}^*}$. The forking argument yields two accepting transcripts with the same R^* and distinct challenges $e_0 \neq e_1$, from which we extract:

$$x_{\mathsf{tag}} := (s_0^* - s_1^*) \cdot (e_0 - e_1)^{-1} = \log_g(\mathsf{vk}_{\mathsf{tag}^*}).$$

Since $\mathsf{vk}_{\mathsf{tag}^*} = \mathsf{vk} \cdot g^{d_{\mathsf{tag}^*}}$ and from the DKG simulation we have $\mathsf{vk} = (\mathsf{vk}^*)^{\tau} \cdot g^{w}$ where $\tau = |I^H_{\mathrm{sel}}|$ is the number of honest physical dealers selected in the DKG and w is known to the simulator, we obtain:

$$x_{\mathsf{tag}} = \tau \cdot x^* + w + d_{\mathsf{tag}^*}.$$

The simulator knows w and can compute $d_{\mathsf{tag}^*} = \mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag}^*)$, so it recovers:

$$x^* = (x_{\mathsf{tag}} - w - d_{\mathsf{tag}^*}) \cdot \tau^{-1} \bmod q.$$

This solves the discrete logarithm problem on vk^*.

If the effective key is the identity, the simulator instead extracts immediately as $x^* = (-d_{\mathsf{tag}^*} - w)\tau^{-1}$. If freshness is defined syntactically by tags rather than effective keys, also account for collisions of the derivation hash: at most $\binom{Q_d}{2}/q$, plus Q_d/q for collisions with the master key, for Q_d derivation evaluations.

Key Observation. The derivation offset $d_{\mathsf{tag}} = \mathsf{H}_{\mathsf{field}}(\mathsf{vk} \| \mathsf{tag})$ is *publicly computable* from the master key vk and the tag tag. This means:

- The simulator can compute d_{tag} for any tag without knowing the secret key.
- The final signature adjustment $\mathsf{H}(m, R, \mathsf{vk}_{\mathsf{tag}}) \cdot d_{\mathsf{tag}}$ can be added by any party.
- Extraction remains efficient because d_{tag^*} is known at forgery time.

Thus, key derivation introduces no additional complexity to the simulation or extraction, and security reduces to the same discrete logarithm assumption as the base protocol.

6.4 Liveness

The composed protocol remains live under an adversary of total weight at most f. This assumes fair authenticated delivery, live TOB/reliable broadcast, eventual disclosure of the required beacon rounds, correct recovery outside the folded-check error, and an epoch long enough to finish the request.

In the AVSS and the batch AVSS, every honest dealer's message eventually reaches all honest parties, which hold total weight at least $2f + 1$ and sign it, so every honest dealer eventually posts a certificate on the TOB. Certified dealings of total weight at least $2f + 1$ therefore accumulate, pre-signing completes, and the complaint procedure supplies valid shares to every honest party. In each signing instance, at least $2f + 1$ honest partials eventually arrive. If $b \leq f$ corrupted responses have also arrived, there are at least $2f + 1 + b \geq f + 1 + 2b$ points, so a decoding attempt correcting b errors

is reachable without waiting for silent corrupted parties. The aggregator eventually outputs a valid signature.

The consistency failure probability is at most $\varepsilon_{\text{fold}}$ as bounded above, in addition to the primitive errors. This bound must be negligible for a cryptographic liveness guarantee; it can be vacuous for large committees even though unforgeability remains covered. Retired slots remain consumed and their messages authorized; restarting uses fresh nonces and beacons. Arbitrarily frequent epoch changes can prevent completion, so this is not an unconditional one-epoch delay guarantee.

6.5 Integration with the AVID sub-protocol

In Section 4 we described how the dealer messages for batch AVSS could be dispersed through the AVID sub-protocol described in Section 7. The same reduction applies with the following bindings and simulation changes:

1. The indexed hashes $e_i = H(\mathsf{sid}, i, E_i)$ bind each complete recipient payload, including the common encryption header and recipient context, before the folding challenges are fixed. The simulator forms the dummy ciphertexts, programs those challenges before publication, and simulates dispersal and complaints. Honest-dealer complaints cannot reveal challenge plaintexts; corrupted-dealer recovery is followed using the actual recipient values.

2. Lemma 3 and the certified common-message digest give one accepted payload per instance. Hash/Merkle binding and reveal-proof errors are included in ε_{aux}. The simulator knows every honest recipient's accepted or recovered corrupted-dealer values, while honest-dealer contributions remain implicit; the pointwise signing argument therefore remains applicable.

In Section 7.2, Lemmas 1 and 2, followed by the checked share-recovery procedure, give liveness subject to the epoch assumptions.

7 Asynchronous Verifiable Information Dispersal with TOB

The batched low-threshold AVSS employed in Hashi involves sending individual elements of an n-sized vector to the respective parties. In this section, we build an efficient mechanism for information dispersal for $W = 3f + 1$.

While the required primitive is similar to Asynchronous Verifiable Information Dispersal (AVID) and can be realized using AVID disperse and retrieve protocols, there are some particular differences: (i) the input message is an n-element vector and, if the sender is honest, every party P_i receives the ith element of the vector at the end of the protocol; (ii) a short proof of correct dispersal should appear on a total order broadcast (TOB) channel, indicating that at least $f + 1$ *honest* parties have received their elements; (iii) it is fine if the rest of the honest parties do not get their element correctly and output $\perp$; however, each of them should be able to prove to the other honest parties that it did not receive a correct element, i.e., they abort in an identifiable manner.

Our required primitive is in fact very similar to the reliable message distribution (RMD) protocol from [SS24], but with two key differences: (i) the RMD protocol may terminate with all parties outputting $\perp$, while we want to terminate the protocol only when at least $f + 1$ honest parties output messages other than $\perp$; (ii) all parties in our setting have access to a TOB channel, and a short protocol termination proof is expected to appear on this TOB channel. Given access to the TOB channel, the issue of equivocation by the sender is resolved trivially.

7.1 Combining with batched VSS

In particular, we combine the reliable message distribution protocol with the batched AVSS flow. Our aim is to reduce the communication overhead as compared to reliable broadcast.

We use the notation with weights considered. We have a dealer D with weight w_D who wants to distribute $w_D L$ nonces to virtual parties in $I^\star = \{(i, v) : i \in I, v \in [1, w_i]\}$ with total weight $W = 3f + 1$. Each virtual party (i, v) has canonical evaluation point $x_{(i,v)}$.

We also include an optimistic phase to reduce the communication overhead. In particular, in this protocol, the dealer sends individual E_i vectors directly to each party and collects signatures from them. In the worst case, it runs the rest of the protocol only for parties of total weight f.

Algorithm 1: Batched AVSS: Optimistic Path

Dealer D:

1. Sample $w_D L$ uniformly random degree-f polynomials $\mathsf{p}_l(X) = c_{l,0} + c_{l,1}X + \cdots + c_{l,f}X^f$ from $\mathbb{Z}_q[X]$ for $l \in [1, w_D L]$.
2. Sample a uniformly random degree-f polynomial $\mathsf{p}'(X) = c'_0 + c'_1 X + \cdots + c'_f X^f$ from $\mathbb{Z}_q[X]$.
3. For each $l \in [1, w_D L]$, compute $C_l \leftarrow g^{c_{l,0}}$.
4. Compute $C' \leftarrow g^{c'_0}$.
5. Compute $r_{l,(i,v)} \leftarrow \mathsf{p}_l(x_{(i,v)})$ for $(i,v) \in I^\star$ and $l \in [1, w_D L]$.
6. Compute $r'_{(i,v)} \leftarrow \mathsf{p}'(x_{(i,v)})$ for $(i,v) \in I^\star$.
7. Create the multi-recipient encryption $E \leftarrow \mathsf{Enc}(\mathsf{sid}, PK^\star, \langle R_{(i,v)} \rangle_{(i,v)\in I^\star})$ where $PK^\star = \{\mathsf{pk}_i : i \in I\}$ (each (i,v) uses pk_i) and $R_{(i,v)} = (r_{1,(i,v)}, \ldots, r_{w_D L,(i,v)}, r'_{(i,v)})$ for $(i,v) \in I^\star$. The output here is represented as an n-sized vector $E = (E_1, \ldots, E_n)$.[a]
8. Compute $e_i = H(\mathsf{sid}, i, E_i) \forall i \in [n]$ and let $e = (e_1, \ldots, e_n)$
9. Compute $\gamma_l \leftarrow \mathsf{H}(\mathsf{sid}, l, \langle C_l \rangle_{l\in[1,w_D L]}, C', e)$ for each $l \in [1, w_D L]$.
10. Compute the polynomial $\mathsf{p}''(X) = \mathsf{p}'(X) + \sum_{l\in[1,w_D L]} \gamma_l \mathsf{p}_l(X)$.
11. Let $CommonMsg \leftarrow D, \mathsf{sid}, E_{\text{com}}, e, \langle C_l \rangle_{l\in[1,w_D L]}, C', \mathsf{p}''(X)$
    Note that the size of E is $O(w_D L * W * \lambda)$, where $w_D L$ is the batch size, W is the total weight in the system, and λ is the computational security parameter. On the other hand, the size of $CommonMsg$ is $O((W + w_D L)\lambda)$
12. For each $j \in [n]$, send $(\texttt{send}, E_j, CommonMsg)$ to P_j.

Every Receiver party P_j**:**

Initialize: `receivedData` $\leftarrow \perp$. `done` $\leftarrow$ `false`; `correctDec` $\leftarrow$ `false`;

wait until either:

if *received* $(\texttt{send}, E_j, CommonMsg)$ *from* D *for the first time* **then**

  Local_Decryption_Processing$(E_j, CommonMsg)$

[a] We use multi-recipient encryption, which contains a common part besides the n ciphertext elements; however, we do not show that, for ease of exposition.

Algorithm 2: Local_Decryption_Processing$(E_i, CommonMsg)$

1. Sanity checks - Check the message length, number of encryptions, that all elements belong to the right groups, that ZK proofs are valid, and that p'' is of the right degree.
2. Verify that $e_i = H(E_i)$
3. For each $l \in [1, w_D L]$, compute $\gamma_l \leftarrow \mathsf{H}(\mathsf{sid}, l, \langle C_l \rangle_{l\in[1,w_D L]}, C', e)$.
4. Verify that $g^{\mathsf{p}''(0)} = C' \cdot \prod_{l\in[1,w_D L]} C_l^{\gamma_l}$.
5. Decrypt the (i,v)-th component of E using $\mathsf{Dec}(\mathsf{sid}, \mathsf{sk}_i, (i,v), E)$. Let $\langle r_{l,(i,v)} \rangle_{l\in[1,w_D L]}, r'_{(i,v)}$ be the decrypted shares.
6. Verify that $\mathsf{p}''(x_{(i,v)}) = r'_{(i,v)} + \sum_{l\in[1,w_D L]} \gamma_l r_{l,(i,v)}$.

if *all checks are valid* **then**

  store the decrypted shares

  `correctDec` $\leftarrow$ `True`

  Send signed $(\texttt{confirm}, H(CommonMsg))$ to D.

else

  Ignore the message

Algorithm 3: Local_Encoded_Message_Processing()

Wait until there exist: some index set $I \subseteq [W]$, $|I| = W - 2f$, and fragments $\{s_{ji}\}_{i\in I}$, such that for all $i \in I$:

- received $(\texttt{echo}, r_j, \pi_{ji}, s_{ji})$,
- π_{ji} is a valid authentication path for s_{ji} under r_j,
- π_j is a valid authentication path for r_j under r.

Reconstruct message E'_j from fragments $\{s_{ji}\}_{i\in I}$.

Encode E'_j as $(s'_{j1}, \ldots, s'_{jn})$ and build a Merkle tree with root r'_j.

Output r'_j.

Collect $CommonMsg$ if it is not available.

Local_Decryption_Processing$(E'_j, CommonMsg)$

Algorithm 4: Batched AVSS: Pessimistic Path

Dealer D: On receiving $(\texttt{confirm}, H(CommonMsg))$ from parties holding $W - f$ equivalent shares or more.

We add an additional wait to collect more than $W - f$ weighted signatures. The ideal case is to collect n signatures from all parties in I.

1. For the remaining f-equivalent parties, $i \in [f]$ (or fewer), encode E_i as $(s'_{i1}, \ldots, s'_{iW})$ using a $(W, W - 2f)$ erasure code, combine those into party shares $(s_{i1}, \ldots, s_{in})$ to give disjoint subset of shares to parties $P_1, \ldots, P_n$ respectively and build a Merkle tree for $(s_{i1}, \ldots, s_{in})$ with root r_i.
2. For each $j \in [n]$, send $(\texttt{send}, \{(r_i, \pi_{ij}, s_{ij})\}_{i=1}^{f}, \texttt{Optimistic Path signatures})$ to P_j, where π_{ij} is a Merkle proof for s_{ij} under r_i.
   Here, we apply the following optimization:
   The dealer D does not send the `send` message to those parties P_i that did not respond with a `confirm` message in the earlier round. However, if a party responds with the optimistic-phase confirmation after the dealer has sent the first round of `send` messages, the dealer forwards the `send` message to every such party.

Every Receiver party P_j**:**

while *not done* **do**

- wait until either:
- **if** *received* $(\texttt{send}, \{(r_i, \pi_{ij}, s_{ij})\}_{i=1}^{f}, \texttt{Optimistic Path signatures})$ *from* D *for the first time* **then**
  - **if** *corresponding optimistic send is not received earlier; i.e.,* $\texttt{correctDec} = \texttt{False}$ **then**
    - discard the message
  - **else**
    - **if** *Verify* $\texttt{Optimistic Path signatures}$ *+ for all rest of those* $i \in [f]$, π_{ij} *is a valid authentication path for* s_{ij} *under* r_i **then**
      - prepare $(\texttt{echo}, r_i, \pi_{ij}, s_{ij})$ for P_i $\forall i \in [f]$, but send only when requested(if n parties confirm in opt path, then no echo msg)
      - Let $r \leftarrow r_i \forall i \in [f]$
      - Send signed $(\texttt{vote}, H(CommonMsg), r)$ to dealer D.
      - Let $\texttt{receivedData} \leftarrow H(CommonMsg), r$
- **if** $(\texttt{vote}, H(CommonMsg), r)$ *from distinct parties with weight* $\geq (W - f)$ *appears on TOB/ABC* **then**
  - **if** $\texttt{receivedData} == H(CommonMsg), r$ **then**
    - $\texttt{done} \leftarrow \texttt{True}$
  - **else**
    - retrieve $CommonMsg$ if not already available
    - request echo messages from the parties who signed the vote messages.
- **if** *received* $(\texttt{echo}, r_j, \cdot, \cdot)$ *from distinct parties with weight* $\geq (W - 2f)$ **then**
  - Local_Encoded_Message_Processing()
  - **if** $r'_j = r_j$ *and* $\texttt{correctDec}$ **then**
    - $\texttt{receivedData} \leftarrow H(CommonMsg), r_i \forall i \in [f]$.
    - $\texttt{done} \leftarrow \texttt{True}$
  - **else**
    - **if** *correctDec* **then**
      - Send all $(\texttt{blame}, (r_j, \pi_{ji}, s_{ji}, H(CommonMsg)) \forall i \in I)$
    - **else**
      - Send all $s_{(i,v)} = Reveal()$ with proof of incorrect decryption

Dealer D**:**

if *received* $(\texttt{vote}, H(CommonMsg), r)$ *from distinct parties with weight* $\geq (W - f)$ **then**

- Forward the aggregate vote from TOB assuming all votes are correctly signed

Every Receiver party P_i**:**

if *(received* $(\texttt{blame}, \cdot)$ *OR* $s_{i,v}$ *from any* P_j*) AND* $\texttt{done}$ **then**

- Verify the blame/Reveal
- Reveal Shares to P_j to allow share recovery

Note that use of the optimistic phase results in a lack of crash recovery for the optimistically sent encryptions. In particular, when all encryptions are sent via reliable broadcast (as we do in Section 4), or when there is no optimistic phase and all encryptions are sent via a protocol akin to the pessimistic phase, any party recovering from a crash can be supplied with its encrypted shares. However, when we employ the optimistic phase, for the parties that send a confirmation in the optimistic phase, recovery of their encrypted shares after a crash may not be possible, as only the dealer and the recovering party

received the encrypted share. For the parties for whom the pessimistic phase was executed, recovery is possible using the echo messages stored by the other parties.

Communication Complexity Comparison In the optimistic phase, the communication overhead is $|E| + n * |CommonMsg|$, ignoring confirmation signatures. Here, $|E| = W * w_D L * \lambda$ and $|CommonMsg| = O(W\lambda + w_D L\lambda)$.

In the pessimistic phase, the communication overhead is reduced to 1/3 of the cost, i.e., $2|E| + \lambda n^2 \log(n)$.

7.2 Safety and Liveness Analyses

We analyze one protocol instance identified by $\tau = (\mathsf{sid}, D)$. For clarity, we first use the unweighted notation $n > 3f$ and assume that at most f parties are Byzantine. The same arguments apply to the weighted protocol by replacing the cardinality of each set by its weight and replacing n by W.

We assume authenticated reliable point-to-point channels between honest parties and a total-order broadcast (TOB) channel.

Let $CommonMsg = (D, \mathsf{sid}, E_{\mathrm{com}}, e, \langle C_l \rangle_l, C', \mathsf{p}'')$, where $e = (e_1, \ldots, e_n)$ and $e_i = H(\mathsf{sid}, i, E_i)$ binds the complete recipient payload. Write $h = H(CommonMsg)$, which already binds τ. A completion certificate has the form $\mathsf{Cert} = (\tau, h, r, S, \sigma_S)$, where r contains the indexed Merkle roots for exactly the recipients outside the initial confirmation set, S is the set of voters, $|S| \geq n - f$, and σ_S is an aggregate signature by the parties in S on (τ, h, r). An honest party votes at most once in an instance τ, and does so only after Local-Decryption-Processing accepts its own element for this same h and after it has received valid authenticated fragments under every root in r. A party is said to complete only after a valid certificate appears on the TOB and the party has accepted its own element, either on the optimistic path or by reconstruction.

Lemma 1 (Honest-sender termination)**.** *If D is honest, then every honest party P_i eventually delivers its VSS shares, and a valid completion certificate eventually appears on the TOB.*

Proof. The honest dealer sends $(E_i, \mathsf{CommonMsg})$ to every P_i. By reliable delivery, every honest party eventually receives this message. Because the dealer formed the ciphertexts, commitments, and polynomial checks correctly, Local-Decryption-Processing accepts at every honest P_i. Hence every honest party stores the decrypted shares and returns a $\mathsf{confirm}$ signature on h.

There are at least $n-f$ honest parties, so the dealer eventually collects enough confirmations without any contribution from Byzantine parties. At most f parties are absent from this confirmation set. The dealer erasure-encodes their elements, distributes valid authenticated fragments to the confirming parties, and requests votes. Every honest confirming party verifies these fragments and returns a vote on the same tuple (τ, h, r). The dealer therefore eventually obtains $n - f$ votes, aggregates them, and posts the resulting certificate on the TOB.

Note that, even if some Byzantine parties are present in the initial confirmation set and they do not send a vote later, the remaining honest parties will come around eventually; i.e., the honest parties that are not in the confirmation set will eventually receive their optimistic-phase messages, send back their confirmations to the dealer, receive the erasure-coded messages from the dealer, verify the correctness of the fragments, decrypt and store the shares, and finally return the vote message to the dealer.

TOB liveness implies that every honest party eventually observes the certificate. Since every honest party already accepted its own E_i and decrypted the encryption to obtain its shares, each then completes with its shares. □

Lemma 2 (Weak totality with identifiable abort)**.** *Suppose an honest party completes an instance. Then at least $f + 1$ honest parties complete with their respective shares. Moreover, every other honest party eventually either (i) reconstructs and delivers its own shares, or (ii) produces a publicly verifiable blame message proving that the dealer dispersed incorrectly.*

Proof. An honest party completes only after observing a valid certificate Cert with at least $n-f$ voters. Since at most f voters are Byzantine, the certificate contains at least $(n - f) - f = n - 2f \geq f + 1$ honest voters. Each such voter signed only after accepting its own shares. After the certificate appears on the TOB, these honest voters complete with their respective elements. Thus completion by one honest party implies completion by at least $f + 1$ honest parties in total.

Now consider any honest party P_i that has not yet completed. By TOB liveness it eventually observes the same certificate and requests the fragments for E_i from the voters. At least $n-2f = f+1$ of the voters are honest, and honest voters eventually return their authenticated fragments. Any $f+1$ such fragments suffice to reconstruct a unique candidate $\widehat{E}_i$ under the $(n, n-2f)$ erasure code.

If re-encoding $\widehat{E}_i$ gives the certified Merkle root r_i and Local-Decryption-Processing accepts $\widehat{E}_i$ relative to $\mathsf{CommonMsg}$, then P_i delivers $\widehat{E}_i$ and continues to the decryption phase. Otherwise P_i constructs blame as follows.

- If re-encoding gives a root $\widehat{r}_i \neq r_i$, the blame message contains Cert and the $n-2f$ authenticated fragments used in reconstruction. Anyone can verify the authentication paths, reconstruct and re-encode the unique candidate, and check $\widehat{r}_i \neq r_i$.
- If the root agrees but the local checks fail, the blame message contains Cert, $\mathsf{CommonMsg}$, $\widehat{E}_i$, and, when the failure concerns decryption, the publicly verifiable reveal proof. A verifier can check either $H(\mathsf{sid}, i, \widehat{E}_i) \neq e_i$ or the failed algebraic/decryption consistency check.

Thus every honest party that cannot deliver eventually obtains finite, publicly verifiable evidence of incorrect dispersal. □

Lemma 3 (No duplication)**.** *Except with negligible probability, every honest party P_i delivers at most one share vector in an instance τ.*

Proof. First, two conflicting completion certificates cannot both be valid. Let S and S' be their voter sets. Since $|S|, |S'| \geq n - f$,

$$|S \cap S'| \geq 2(n-f) - n = n - 2f > f.$$

As at most f parties are Byzantine, $S \cap S'$ contains an honest party. That party would have had to vote for two different tuples in the same instance, contradicting the rule that an honest party votes at most once (or else yielding a signature forgery).

It remains to consider two values accepted relative to the same certificate. Both the optimistic and recovery paths check the index-specific commitment $e_i = H(\mathsf{sid}, i, E_i)$ contained in the certified $\mathsf{CommonMsg}$. Therefore, if P_i accepted two distinct values $E_i \neq E'_i$, then $H(\mathsf{sid}, i, E_i) = H(\mathsf{sid}, i, E'_i) = e_i$, giving a collision in H. On the recovery path, the Merkle root additionally binds the authenticated fragments to a unique reconstructed codeword. Hence, except with negligible probability of breaking one of these binding assumptions, the two accepted encryption values are identical. Consequently, the two accepted share vectors are also identical. □

Note that the TOB certificate is a proof that at least $f+1$ honest parties received and accepted their designated elements, and created verifiable fragments in the form of the echo messages. Therefore, the presence of the certificate guarantees termination when the dealer is Byzantine.

8 Flexible threshold modifications

We describe modifications to support general thresholds t beyond the baseline $n = 3f+1$ regime. Let n be the committee size and at most f parties be Byzantine. In the weighted setting of Section 3, n denotes the total weight W, the Byzantine bound f is a bound on total corrupted weight, and every party count below is read as a total weight. We target a signing threshold t satisfying

$$t + 2f \;\leq\; n\,, \qquad t > f\,.$$

Modified DKG (full completeness). Unless noted, all steps remain the same as in Section 3.

- Dealer sampling: use degree-$(t-1)$ polynomials (instead of degree-f).
- Dealer broadcast: remains the same.
- Acks/fraud proofs to TOB: remains the same.

- Waiting condition: parties wait for t dealer messages (instead of $f{+}1$). Among these, at most f are adversarial, so at least one is honest. Similarly, parties wait for $t+f$ confirmation messages (instead of $2f+1$), so at least t are from honest parties.
- Output: each party locally outputs the joint vk and its share if any; remains the same modulo degree $(t{-}1)$.
- Ongoing recovery: parties with valid shares help others recover upon valid fraud proofs; remains the same.

Assumption. We require $t{+}2f \leq n$ so that the system can always collect $t{+}f$ confirmation messages including at least t honest ones. [3]

Modified batch nonce generation. Unless noted, all steps remain the same as in Section 4.

- Dealer polynomials: use degree-$(t{-}1)$ for $\{\mathsf{p}_l\}$ and for p'. Commitments and ciphertext layout remain the same.
- RO constraint: remains the same (program or query $\gamma_l = \mathsf{H}(\cdot)$ and set $\mathsf{p}'' = \mathsf{p}' + \sum_l \gamma_l \, \mathsf{p}_l$).
- Certification threshold: require $t{+}f$ signatures on the broadcast tuple (instead of $2f{+}1$). This ensures at least t honest signers certify the tuple and have valid shares.
- Fraud handling and reveal: remains the same.

Modified presigning (SI matrix). Unless noted, all steps remain the same as in Section 5.

- Let J be the set of certified dealers and $W_J = \sum_{j \in J} w_j$ their total weight. We wait until $W_J \geq n - f$ (so that $W_J \geq t$, since $n - f \geq t + f$), and optionally accumulate more certificates already posted on the TOB, up to $W_J = n$.
- We choose $M := W_J - (t-1)$ so that we derive on the order of M independent presign commitments per column. The bound here is the privacy threshold $t-1$ (the degree of the nonce sharings), not the Byzantine bound f: a sub-t coalition may control dealers of total weight up to $t-1$ and hence know or bias $t-1$ of the W_J input nonces, so only $W_J - (t-1)$ extracted nonces remain uniform. As W_J ranges from the $n-f$ floor up to n, $M = W_J - (t-1)$ ranges from $n - f - (t-1) \geq f+1$ (using $t + 2f \leq n$) up to $n - (t-1)$.
- The simulator in the proof is modified in the following way:
  - Form the commitment matrix $C \in \mathbb{G}^{W_J \times L}$ and auxiliary arrays as before. Let $S \in \mathbb{F}^{M \times W_J}$ be a super-invertible matrix (any M of its W_J columns are linearly independent; in particular each row has more than $t-1$ nonzero entries).
  - Compute
$$D := S \cdot C\,, \quad U := S \cdot U'\,, \quad V := S \cdot V'\,, \quad P := S \cdot P'\,.$$
As before in the simulation, $D = g^U \cdot \mathsf{vk}^V$. A sub-$t$ adversary knows at most $t-1$ of the W_J columns of V', leaving at least $W_J - (t-1) = M$ columns outside its view; because S is super-invertible the corresponding $M \times M$ submatrix is nonsingular, so every row of V is uniformly random in $\mathbb{Z}_q^L$ from the adversary's view. This guarantees that programming the RO at (vk, R, m) to $-V_{\rho,l}$ remains indistinguishable from lazy sampling in the simulation.

Modified online signing and aggregation. Unless noted, all steps remain the same as in Section 5.

- Threshold: the aggregator accepts when it collects t valid partials for a coordinate; remains the same algorithmically with t in place of $f{+}1$.
- RS decoding: set code dimension $k = t$; the aggregator can correct up to $\lfloor (N-t)/2 \rfloor$ errors out of N received partials at that coordinate. The loop that increases tolerance with more partials remains the same with t substituted.

[3] The inequality ensures quorum availability. Section 9 replaces this availability argument by the coalition conditions (L1)–(L2); it is not needed to count the honest members of an already formed certificate.

Modified resharing. Unless noted, all steps remain the same as in Section 3.2.

- Dealer polynomials during resharing use degree $(t-1)$, with $C_{i,0}$ fixed to the existing partial key as before.
- Waiting condition and certification use $t+f$ in place of $2f+1$.
- Completeness goal remains: ensure at least $n-f$ honest parties end up with valid refreshed shares so that subsequent rounds can succeed for threshold t.

Modified AVID dispersal. Steps in Section 7 are aligned with flexible thresholds. In particular,

- Optimistic path: the dealer waits for confirmations from virtual parties of total weight $n - f$ before entering the pessimistic path.
- Pessimistic path: the dealer waits for votes from virtual parties of total weight $n - f$.
- Erasure code: encode each pending element with an $(n, n-2f)$ code. Collection and reconstruction use fragments of total weight $n - 2f$.

9 Knapsack Weight Reduction

In production deployments, the signing committee is rarely uniform: each party i holds an integer stake w_i and is entitled to w_i independent shares, so the effective committee size is the total stake $W = \sum_{i=1}^{n} w_i$ rather than the number of physical parties. Plugging W directly into the constructions of Sections 3 through 5 is costly: dealer polynomials of degree $t-1$, super-invertible matrices of dimension $M \times W_J$, and Reed-Solomon decoding all scale polynomially in the committee size, and realistic stake distributions push W into the tens of thousands. In this section, we work in a reduced space by introducing a small liveness slack δ.

The problem. For Hashi, total stake weight is W, which can be close to $10,000$. Here, we want to run the whole protocol with much smaller integer weights w_i', total W' of a few hundred, and smaller thresholds t', f', preserving the reconstruction access structure while paying the stated liveness slack.

The challenge is that rounding is not linear: a coalition that is just below the threshold in real stake can jump above it after rounding, and vice versa, and the set of coalitions that matters is every subset of parties.

Our Approach For the original privacy threshold t, Byzantine bound f and with the total weight $W = t + 2f$, we introduce a liveness slack δ (a small fraction of the stake). Let $w(S)$ and $w'(S)$ be the original and reduced weight of a coalition/subset S respectively.

We call (w', t', f') *admissible* if for every subset S following four conditions hold:

(P) $w(S) < t \Rightarrow w'(S) < t'$: nobody who could not reconstruct before can reconstruct now (privacy, hence unforgeability);

(L1) $w(S) \geq t + f + \delta \Rightarrow w'(S) \geq t' + f'$: a set big enough to be a Byzantine-tolerant quorum before is still one;

(L2) $w(S) \leq f \Rightarrow w'(S) \leq f'$: the adversary gains no reduced weight beyond f';

(L3) $w(S) \geq t + \delta \Rightarrow w'(S) \geq t'$: a set that could reconstruct before (with a bit of slack) still can.

Note that condition (L3) is not a consequence of the other two liveness conditions: it protects sets that arise in odd ways. For example, the share-holders that remain after some crashes, when the parties outside the set are few and heavy.

Here, δ is the price of rounding, paid in liveness only. With the honest set H of stake $W - b$ when the adversary has stake b:

- *Secrecy against* $t - 1$. If $b \leq t - 1$ then (P) gives the adversary reduced weight $< t'$.

- *Liveness against $f - \delta$.* If $b \le f - \delta$ then $w(H) \ge t + f + \delta$, so by (L1) honest parties alone reach every $t' + f'$ quorum (DKG confirmations, certificates, presigning), and by (L2) any such quorum contains honest reduced weight $\ge t'$.
- *Key material survives losing stake f.* After a DKG run with $b \le f - \delta$, honest share-holders have stake $\ge t + f + \delta$; if stake up to f disappears (crashed or Byzantine), what is left has stake $\ge t + \delta$, so by (L3) reduced weight $\ge t'$ and can still sign and reshare.
- *Signing.* With $b \le f - \delta$ the aggregator gets correct partials of reduced weight $\ge t' + f'$ (L1) and faulty ones of reduced weight $e \le f'$ (L2); Reed–Solomon decoding needs $t' + 2e$ partials, and $t' + f' + e \ge t' + 2e$, so it succeeds.

Checking a candidate. While the above four conditions quantify over all subsets, each one asks for the *extreme* coalition of a given original weight, and that is a knapsack question.

Let $g(b) = \max\{w'(S) : w(S) \le b\}$, the most reduced weight a coalition of stake $\le b$ can have. Now, for condition (P) says $g(t-1) \le t' - 1$, condition (L2) says $g(f) \le f'$, and the two liveness conditions become statements about complements: the *least* reduced weight of a set with $w(S) \ge t + \delta$ is $W' - g(W - t - \delta)$, which must be $\ge t'$, and similarly $W' - g(W - t - f - \delta) \ge t' + f'$.

We can compute the function g by the classic value-space knapsack recursion in $O(nW')$ time.

Notes regarding existing protocol and analysis. First, use $t' + f'$ for VSS certification and dealer accumulation, and the AVID quorum specified below. A $W' - f'$ certificate alone may contain fewer than t' honest members when $t' + 2f' > W'$.

Second, the reduced-space liveness argument must cite (L1)/(L2) rather than the $t + 2f \le n$ inequality of the flexible-threshold section. The security argument remains unchanged, it only needs $f' < t'$ and the Byzantine bound f'.

Finding a small one following Swiper. Candidates are scaled roundings $w'_i = \lfloor w_i/d + c \rfloor$ with a divisor d on a 0.01 grid and a rounding offset $c \in \{0, 0.1, \ldots, 0.9\}$ ($c = 0$ floors, $c = 0.5$ rounds). For a fixed offset the total W' only grows as d shrinks, so we sweep d *downward* from the largest useful value $\max_i w_i/(1-c)$ and stop at the first admissible candidate, which is the cheapest one for that offset. Binary search would be wrong, because admissibility is not monotone in d: it depends on the rounding pattern, not just on the scale. Two tricks keep this fast. Consecutive grid points usually give the same vector, so the sweep jumps straight to the next breakpoint, the largest d at which some w'_i ticks up (party i goes from q to $q+1$ at $d = w_i/(q+1-c)$). And most candidates are thrown out before the $O(nW')$ check by a greedy lower bound on g: sort parties by density w'_i/w_i, take what fits; this underestimates g by less than one party's reduced weight, and if even the underestimate violates the rearranged liveness conditions the candidate is certainly bad. The best result over the ten offsets wins (fewest shares, then smallest t'); if nothing is admissible the identity is returned. Everything runs in 64-bit integer arithmetic and takes well under a second for $n \le 1000$, $W \le 10^4$. The design follows Swiper [TF24] (scaled rounding verified by knapsack), with tight output thresholds, an offset grid, and a downward sweep in place of binary search.

9.1 Updates to AVID due to Weight Reductions

After weight reduction, the system is only expected to live with additional δ liveness slack. As a result, we need to update protocol parameters correspondingly.

In particular, for AVID protocol, after weight reduction, $W' - f'$ can be lower than $t' + f'$. Therefore, all $W - f$ should be replaced by $\max(W' - f', t' + f')$. Moreover, no duplication lemma for AVID that replies on the quorum intersection may be hold either. We will instead rely on the TOB channel to ensure that only message succeeds. For every AVID instance, we simply use the first correct certificate and ignore the rest.

References

[ABCP23] Shahla Atapoor, Karim Baghery, Daniele Cozzo, and Robi Pedersen. VSS from distributed ZK proofs and applications. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 405–440. Springer, Singapore, December 2023.

[BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532. Springer, Berlin, Heidelberg, December 2001.

[Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.

[DFI24] DFINITY Foundation. The Internet Computer interface specification. https://internetcomputer.org/docs/references/ic-interface-spec, 2024.

[GS22] Jens Groth and Victor Shoup. Design and analysis of a distributed ECDSA signing service. Cryptology ePrint Archive, Report 2022/506, 2022.

[HJKY95] Amir Herzberg, Stanislaw Jarecki, Hugo Krawczyk, and Moti Yung. Proactive secret sharing or: How to cope with perpetual leakage. In Don Coppersmith, editor, *CRYPTO'95*, volume 963 of *LNCS*, pages 339–352. Springer, Berlin, Heidelberg, August 1995.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[MR21] Atsuki Momose and Ling Ren. Multi-threshold byzantine fault tolerance. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 1686–1699. ACM Press, November 2021.

[OY91] Rafail Ostrovsky and Moti Yung. How to withstand mobile virus attacks (extended abstract). In Luigi Logrippo, editor, *10th ACM PODC*, pages 51–59. ACM, August 1991.

[Sch90] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 239–252. Springer, New York, August 1990.

[SG98] Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In Kaisa Nyberg, editor, *EUROCRYPT'98*, volume 1403 of *LNCS*, pages 1–16. Springer, Berlin, Heidelberg, May / June 1998.

[Sho25] Victor Shoup. The many faces of Schnorr: A toolkit for the modular design of threshold Schnorr signatures. *IACR Communications in Cryptology*, 2(1), 2025.

[Sho26] Victor Shoup. The many faces of Schnorr: A touch-up. Cryptology ePrint Archive, Paper 2026/681, https://eprint.iacr.org/2026/681, 2026.

[SJSW19] Philipp Schindler, Aljosha Judmayer, Nicholas Stifter, and Edgar Weippl. ETHDKG: Distributed key generation with Ethereum smart contracts. Cryptology ePrint Archive, Report 2019/985, 2019.

[SS01] Douglas R. Stinson and Reto Strobl. Provably secure distributed Schnorr signatures and a (t, n) threshold scheme for implicit certificates. In Vijay Varadharajan and Yi Mu, editors, *ACISP 01*, volume 2119 of *LNCS*, pages 417–434. Springer, Berlin, Heidelberg, July 2001.

[SS24] Victor Shoup and Nigel P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *Journal of Cryptology*, 37(3):27, July 2024.

[TF24] Andrei Tonkikh and Luciano Freitas. Swiper: a new paradigm for efficient weighted distributed protocols. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing (PODC 2024)*. ACM, 2024.

A Reed-Solomon Decoding

We considered several candidate RS decoding algorithms - the Berlekamp-Welch algorithm, the Berlekamp-Massey algorithm, and Gao's extended-Euclidean-based algorithm - comparing their asymptotic complexity and concrete efficiency. We first recall the common structure underlying all of them, then discuss each in turn, and refer to an open-source library.

Structure of Reed-Solomon Error Correcting Algorithms

1. Consider an $(n, f+1)$ RS codebook and let p be a degree f polynomial.
2. Consider evaluation points $[0, n-1]$
3. We are given evaluation vector $\langle b_0, \ldots, b_{n-1} \rangle$ out of which there are e errors
4. Implicitly assume the (unknown) locations of the errors are $e'_0, \ldots, e'_{e-1}$
5. The error-locator polynomial is defined as

$$\Lambda(x) = \prod_{i=0}^{e-1} (x - e'_i)$$

   Denote $\Lambda(x) = \Lambda_0 + \Lambda_1 x + \ldots + \Lambda_{e-1} x^{e-1} + x^e$, in terms of its coefficients.
6. Observe that $b_i \Lambda(i) = p(i)\Lambda(i)$ for $i = 0, \ldots, n-1$
7. We have e unknowns on the left hand side and $e+f+1$ unknowns on the right hand side (the coefficients of the degree-$(e+f)$ polynomial $p(x)\Lambda(x)$), for a total of $2e+f+1$ unknowns. Hence it can be solved uniquely if $n \geq 2e+f+1$.

Berlekamp-Welch algorithm

1. A naive version of the BW algorithm simply solves the system of equations for each e from 1 to $\lfloor (n-f-1)/2 \rfloor$.
2. The time complexity is $O(n^{3-\epsilon})$ for solving the system of equations. This is repeated for each e from 1 to $\lfloor (n-f-1)/2 \rfloor$.
3. The total time complexity is $O(n^{4-\epsilon})$ for $f, e \in O(n)$.
4. However, observe that we could homogenize the system, instead of setting $\Lambda_e = 1$.
5. This way, any solution of the system of equations with $e' < e$ is also a solution of the system of equations with e.
6. This means that we can solve the system of equations for $e = \lfloor (n-f-1)/2 \rfloor$ and then use the solution to solve the system of equations for $e = \lfloor (n-f-1)/2 \rfloor - 1$.
7. This way, we can solve the system of equations for e from $\lfloor (n-f-1)/2 \rfloor$ to 1 in $O(n^{3-\epsilon})$ time.
8. The updated time complexity is $O(n^{3-\epsilon})$ for $f, e \in O(n)$.

Berlekamp-Massey algorithm

1. The Berlekamp-Massey algorithm is a more efficient way to solve the system of equations for e from $\lfloor (n-f-1)/2 \rfloor$ to 1.
2. The BM algorithm observes that the codeword $\langle E(i) b_i \rangle_{i=0}^{n-1}$ corresponds to a polynomial of degree $e+k-1$ in i.
3. Therefore, $\langle E(i) b_i \rangle_{i=0}^{n-1}$ is orthogonal to the dual code $RS[a_{[n]}, n, n-k]$.
4. Specifically for if a_i are the roots of unity in a prime field F_q, then $RS[a_{[n]}, n, n-k] = Span(EvalF(x^i))$.

5. Then $\sum_{j=0}^{e} \sum_{i=0}^{n-1} E_j \alpha^{ij} b_i \alpha^{il} = 0$ for $l \in [1, n-t-k]$.
6. This means that $\sum_{j=0}^{e} \sum_{i=0}^{n-1} E_j b_i \alpha^{i(j+l)} = 0$ for $l \in [1, n-t-k]$.
7. BM showed how to solve this system of equations in $O(n^2)$ time.

Gao's algorithm

1. Let $g(x) = \prod_{i=0}^{n-1}(x - a_i)$ be the zero polynomial for the evaluation points $a_0, \ldots, a_{n-1}$.
2. Let $b(x)$ be the polynomial of degree at most $n-1$ that interpolates (a_i, b_i).
3. Observe that $b(X)E(X) - p(X)E(X)$ is 0 at $a_0, \ldots, a_{n-1}$.
4. This means that $b(X)E(X) - p(X)E(X)$ is divisible by $g(X)$.
5. This means that $b(X)E(X) - p(X)E(X) = g(X)W(X)$ for some polynomial $W(X)$.
6. Solve for low degree polynomials (W, E, N) such that $g(X)W(X) + b(X)E(X) = N(X)$ by using extended Euclidean algorithm. This is done in $O(n^{2-\epsilon})$ time, or $O(n \log n)$ time with more sophisticated algorithms.
7. The message polynomial $p(x)$ is then given by $p(x) = N(x)/E(x)$.

Observations

1. BM and Gao have similar time complexity.
2. Both enjoy better concrete efficiency when the evaluation points are a complete set of roots of unity.
3. In BM, the dual code is another RS code of simple description which makes the linear equations simpler to set up.
4. In Gao, the zero-polynomial is very simple when the evaluation points are a complete set of roots of unity.

The libcorrect library. The libcorrect library[4] implements Reed-Solomon error correction algorithms using the following algorithms:

1. Berlekamp-Massey algorithm for finding the error locator polynomial.
2. Chien's algorithm for finding the roots of the error locator polynomial.
3. Forney algorithm for finding the error values.

However, the libcorrect library does not implement prime fields and arbitrary sequence of evaluation points.

[4] https://github.com/libcorrect/libcorrect